

DOSSIER PROFESSIONNEL (DP)

Nom de naissance

▶ PAQUIER

Nom d'usage

▶ PAQUIER

Prénom

▶ Mathieu

Adresse

▶ 40 boulevard de Sarrebruck, appt A702, 44000 NANTES

Titre professionnel visé

Cliquez ici pour entrer l'intitulé du titre professionnel visé.

MODALITE D'ACCES :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.
Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- ▶ pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité

p.

5

- ▶ Maquetter une application p.
- ▶ Réaliser une interface utilisateur web statique et adaptable..... p.
- ▶ Développer une interface utilisateur web dynamique p.
- ▶ Réaliser une interface utilisateur avec une solution de gestion de contenu ou e-commerce ... p.

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité

p.

- ▶ Créer une base de données p.
- ▶ Développer la partie back-end d'une application web ou web mobile..... p.
- ▶ Élaborer et mettre en œuvre des composants de gestion dans une application e-commerce .. p.

Titres, diplômes, CQP, attestations de formation *(facultatif)*

p.

Déclaration sur l'honneur

p.

Documents illustrant la pratique professionnelle *(facultatif)*

p.

Annexes *(Si le RC le prévoit)*

p.

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n°1 ► Maquetter une application

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

1. Présentation du projet

La région Auvergne Rhône-Alpes m'a contacté pour la création de la partie frontend d'une plateforme dédiée aux artisans de la région : **Trouve ton artisan**.

L'objectif est de permettre aux particuliers de trouver un artisan et de lui demander facilement des renseignements, prestations ou encore tarifs via un formulaire de contact.

La région m'a fournis une charte graphique à respecter comprenant :

- Couleurs :
- Logo
- Typographie

Pour réaliser ce projet, j'ai utilisé l'outil de conception d'interface Figma, qui m'a permis de créer des maquettes d'interfaces web.

2. Mise en place des maquettes

J'ai réalisé les maquettes graphiques du site. C'est une représentation visuelle détaillée d'une interface web.

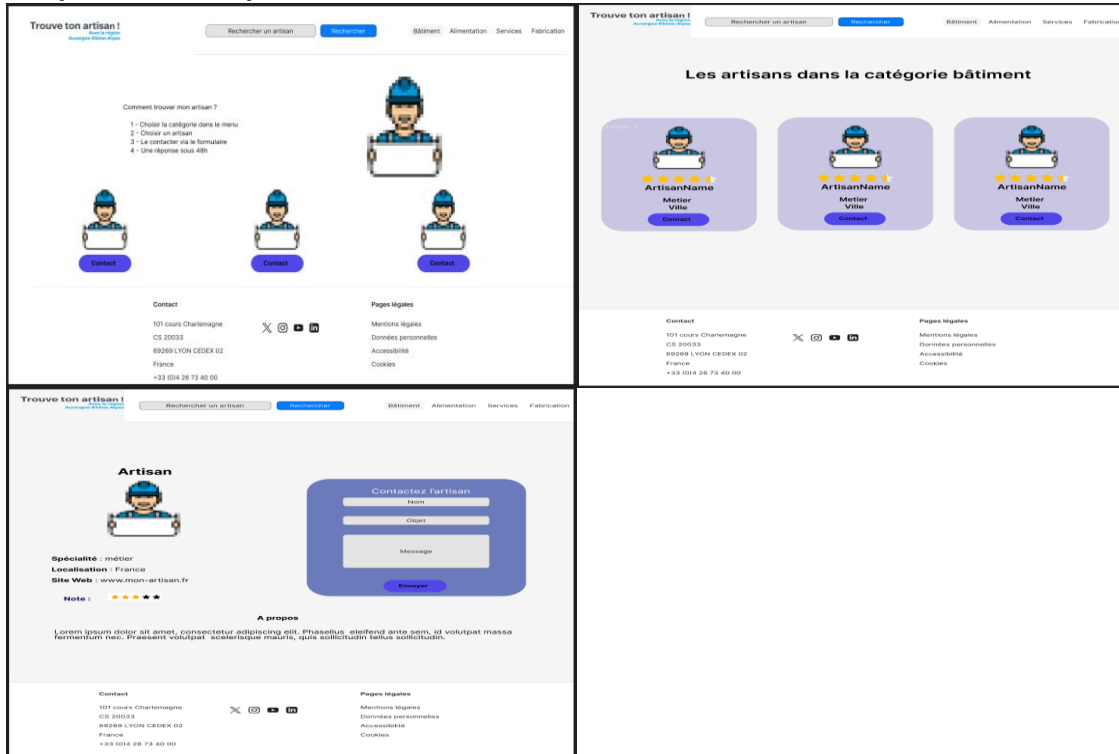
Cela permet d'obtenir une vision plus proche du rendu final du site internet. J'ai donc appliqué la charte graphique fournie par la région.

Les pages du site sont les suivantes :

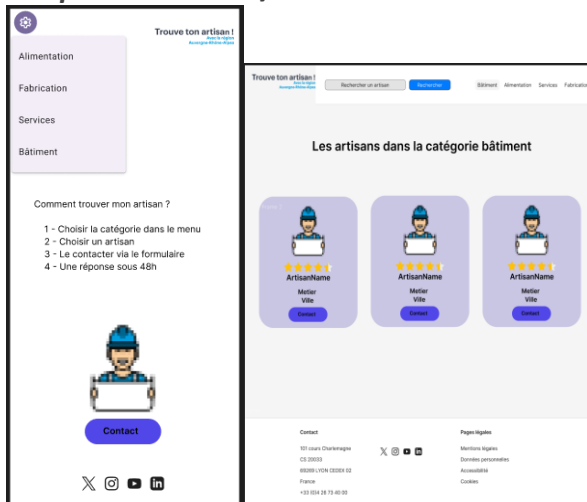
- Page d'accueil
- Page liste d'artisan selon catégorie
- Page fiche artisan/contact

DOSSIER PROFESSIONNEL (DP)

Maquettes desktop



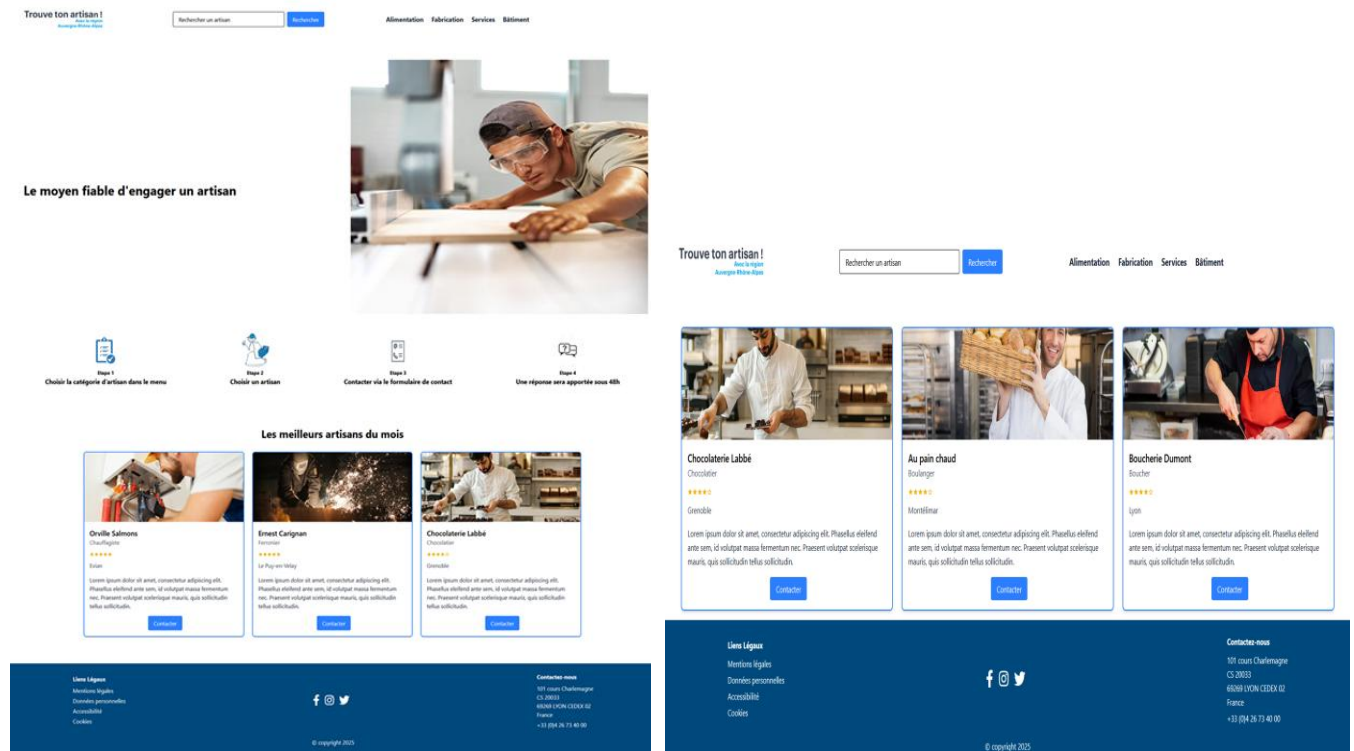
Maquettes mobiles/tablettes



DOSSIER PROFESSIONNEL (DP)

Rendu du site internet

Version desktop :



Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

Rechercher un artisan

Rechercher

Alimentation Fabrication Services Bâtiment



Chocolaterie Labbé

Chocolatier

★★★★★

Grenoble

chocolaterie-labbe@gmail.com

https://chocolaterie-labbe.fr

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus eleifend ante sem, id volutpat massa fermentum nec. Praesent volutpat scelerisque mauris, quis sollicitudin tellus sollicitudin.

Contactez-moi

Nom

Votre nom

Email

Votre email

Message

Votre message

Envoyer

Liens Liéaux

Mentions légales
Données personnelles
Accessibilité
Cookies



Contactez-nous

101 cours Charlemagne
CS 20033
69269 LYON CEDEX 02
France
+33 (0)4 26 73 40 00

© copyright 2025

DOSSIER PROFESSIONNEL (DP)

Version mobile :

Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

Rechercher

Le moyen fiable d'engager un artisan



Etape 1
Choisir la catégorie d'artisan dans le menu

Etape 2
Choisir un artisan

Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

Rechercher

Chocolaterie Labbé

Chocolatier

★★★★☆

Grenoble

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus eleifend ante sem, id volutpat massa fermentum nec. Praesent volutpat scelerisque mauris, quis sollicitudin tellus sollicitudin.

Contactez

Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

Rechercher

Chocolaterie Labbé

Chocolatier

★★★★☆

Grenoble

chocolaterie-labbé@gmail.com

https://chocolaterie-labbé.fr

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus eleifend ante sem, id volutpat massa fermentum nec. Praesent volutpat scelerisque mauris, quis sollicitudin tellus sollicitudin.

Contactez-moi

Nom

Votre nom

Email

Votre email

Message

Votre message

Envoyer

Liens Légaux

Mentions légales

Données personnelles

Accessibilité

Cookies

f i t

Contactez-nous

101 cours Charlemagne
CS 20033
69269 LYON CEDEX 02
France
+33 (0)4 26 73 40 00

© copyright 2025

Version tablette :

Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

Rechercher un artisan

Rechercher

Le moyen fiable d'engager un artisan



Etape 1
Choisir la catégorie d'artisan dans le menu

Etape 2
Choisir un artisan

Etape 3
Contacter via le formulaire de contact

Etape 4
Une réponse sera apportée sous 48h

Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

Rechercher un artisan

Rechercher

Chocolaterie Labbé

Chocolatier

★★★★☆

Grenoble

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus eleifend ante sem, id volutpat massa fermentum nec. Praesent volutpat scelerisque mauris, quis sollicitudin tellus sollicitudin.

Contactez

Trouve ton artisan !
Avec la région
Auvergne-Rhône-Alpes

Rechercher un artisan

Rechercher

Chocolaterie Labbé

Chocolatier

★★★★☆

Grenoble

chocolaterie-labbé@gmail.com

https://chocolaterie-labbé.fr

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus eleifend ante sem, id volutpat massa fermentum nec. Praesent volutpat scelerisque mauris, quis sollicitudin tellus sollicitudin.

Contactez-moi

Nom

Votre nom

Email

Votre email

Message

Votre message

Envoyer

Liens Légaux

Mentions légales

Données personnelles

Accessibilité

Cookies

f i t

Contactez-nous

101 cours Charlemagne
CS 20033
69269 LYON CEDEX 02
France
+33 (0)4 26 73 40 00

© copyright 2025

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Outil de conception de maquettage Figma.

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre européen de formation*

Chantier, atelier, service ► **Projet : Trouve ton artisan**

Période d'exercice ► Du : *Cliquez ici* au : *Cliquez ici*

5. Informations complémentaires (facultatif)

Ce projet m'a permis de découvrir et de prendre en main Figma, un outil utilisé dans le domaine de la conception d'interfaces web.

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n°2 ► *Réaliser une interface utilisateur web statique et adaptable*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

I. Réaliser une interface utilisateur web

Ce projet étant conçu pour permettre aux particuliers de trouver un artisan et de lui demander facilement des renseignements, prestations ou encore tarifs via un formulaire de contact. Il était nécessaire d'avoir une interface simple et facile d'utilisation, peu importe le niveau de compétence en informatique.

Résumé du projet :

La région Auvergne-Rhône-Alpes couvre 12 départements dans le quart sud-est de la France. Elle a des bureaux à Lyon et à Clermont-Ferrand. Vous êtes en relation avec ceux de Lyon.

Près d'un tiers des entreprises de la région sont des entreprises d'artisanat. On en comptait 221000 en 2021. C'est l'une des régions les plus artisanales de France ! Et la région compte bien entretenir cet engouement, d'où la création de la plateforme.

Cette application va donc faciliter la recherche d'artisans, de leur demander facilement des renseignements, prestations ou encore tarifs via un formulaire de contact.

II- Spécifications techniques

A. Technologies

➤ Gestionnaire de paquets et developper experience

1. NPM
2. Eslint
3. Prettier

➤ Front-end

1. Angular
2. TailwindCSS
3. Angular-router
4. EmailJS
5. GitHub pages
6. VSCode

Compatibilité navigateurs :

Selon une étude de marché de 2024, les navigateurs les plus utilisés sont :

- Sur desktop
 - o Chrome : 57,13 %
 - o Firefox : 16,36 %
 - o Edge : 11,74 %
- Sur mobile
 - o Chrome : 56,03 %
 - o Safari : 30,15 %

Notre application sera donc compatible avec les versions les plus récentes des navigateurs cités ci-dessus.

B. Les pages

1. Architecture du Projet

Framework Frontend : Angular

- o Utilisation d'Angular pour la création d'une application monopage (SPA) permettant une navigation fluide et rapide entre les différentes vues.
- o Structure modulaire avec des composants réutilisables.

CSS Framework : Tailwind CSS

- o Utilisation de Tailwind CSS pour le styling, permettant une personnalisation rapide et efficace des composants UI.
- o Approche utility-first pour un développement plus rapide et une maintenance simplifiée.

2. Composants Principaux

Header et Footer

- o Header : Inclut un logo, une barre de recherche, et un menu de navigation responsive.
- o Footer : Contient des liens légaux, des icônes de réseaux sociaux, et des informations de contact.

Pages Principales

- o Accueil : Présentation des meilleurs artisans et étapes pour les contacter.
 - o Liste des Artisans : Affichage des artisans par catégorie.
 - o Détail d'un Artisan : Fiche détaillée d'un artisan avec un formulaire de contact.
 - o Recherche : Fonctionnalité de recherche d'artisans.
 - o Page 404 : Page d'erreur personnalisée pour les routes non trouvées.
- Fonctionnalités Clés

Recherche d'Artisans

- o Implémentation d'une barre de recherche permettant de filtrer les artisans par nom, spécialité, ou localisation.

- o Utilisation d'un service de recherche pour gérer les résultats de recherche.

Recherche via les Catégories

Fonctionnement :

- o Les catégories d'artisans sont accessibles via le menu de navigation dans le header.

- o Chaque catégorie est associée à une route spécifique (par exemple, /artisans/Alimentation).

- o Lorsque l'utilisateur clique sur une catégorie, la route correspondante est activée, et le composant CategoryComponent est chargé.

Implémentation Technique :

- o Angular Router : Utilisation du module RouterModule pour configurer les routes.

- o ActivatedRoute : Utilisation du service ActivatedRoute pour récupérer les paramètres de la route (par exemple, la catégorie sélectionnée).

- o Data Service : Utilisation du service DataService pour récupérer les données des artisans depuis un fichier JSON.

- o Filtrage des Données : Filtrage des artisans en fonction de la catégorie sélectionnée.

```
export class CategoryComponent implements OnInit {  
  artisans: any[] = [];  
  category: string = '';  
  
  constructor(  
    private route: ActivatedRoute,  
    private dataService: DataService  
  ) {}  
  
  ngOnInit(): void {  
    this.route.params.subscribe((params) => {  
      this.category = params['category'];  
      this.loadArtisans();  
    });  
  }  
  
  loadArtisans(): void {  
    this.dataService.getData().subscribe((data) => {  
      this.artisans = data.filter(  
        (artisan) => artisan.category === this.category  
      );  
    });  
  }  
}
```

Barre de Recherche

Fonctionnement :

- o La barre de recherche permet à l'utilisateur de rechercher des artisans par nom, spécialité, ou localisation.
- o Lorsque l'utilisateur saisit un terme de recherche, les résultats sont filtrés et affichés en temps réel.

Implémentation Technique :

- o Événement input : Utilisation de l'événement (input) pour détecter les changements dans le champ de recherche.
- o Filtrage des Données : Filtrage des artisans en fonction du terme de recherche saisi.
- o Search Service : Utilisation du service SearchService pour gérer les résultats de recherche et les mettre à jour de manière réactive.

```
onSearch(event: Event): void {
  const input = event.target as HTMLInputElement;
  this.searchTerm = input.value.toLowerCase();

  if (this.searchTerm.trim() === '') {
    this.searchService.updateSearchResults([]);
    this.Router.navigate(['/']);
  } else {
    const filteredArtisans = this.artisans.filter(artisan =>
      artisan.name.toLowerCase().includes(this.searchTerm) ||
      artisan.specialty.toLowerCase().includes(this.searchTerm) ||
      artisan.location.toLowerCase().includes(this.searchTerm)
    );
    this.searchService.updateSearchResults(filteredArtisans);
    this.Router.navigate(['/search']);
  }
}

// Function to clean up the subscription and prevent memory leaks
ngOnDestroy(): void {
  if (this.searchSubscription) {
    this.searchSubscription.unsubscribe();
  }
}
```

Construction des Fiches Artisans

Fonctionnement :

- o Chaque artisan est représenté par une fiche détaillée contenant des informations telles que le nom, la spécialité, la localisation, l'email, le site web, et une description.
- o Les fiches artisans sont affichées dans une grille responsive sur les pages de liste des artisans et de recherche.

Implémentation Technique :

- o Composant ArtisanDetailComponent : Utilisation d'un composant dédié pour afficher les détails d'un artisan.
- o Data Service : Utilisation du service DataService pour récupérer les données des artisans.
- o Angular Router : Utilisation du module RouterModule pour naviguer vers la page de détail d'un artisan.

Chargement des données des artisans :

```
loadArtisan(id: string): void {  
  this.dataService.getData().subscribe((data) => {  
    this.artisan = data.find((artisan) => artisan.id === id);  
  });  
}
```

Template :

```
<div class="container mx-auto p-4">
  <div class="flex flex-col md:flex-row">
    <!-- Detailed artisan's card -->
    <div class="w-full md:w-1/2 p-4">
      <img
        [src]="artisan.img"
        alt="{{ artisan.name }}"
        class="w-full h-64 object-cover"
      />
      <h1 class="text-2xl font-bold mt-4">{{ artisan.name }}</h1>
      <p class="text-gray-600">{{ artisan.specialty }}</p>
      <div class="flex items-center mt-2">
        @for (i of [1, 2, 3, 4, 5]; track i) {
          <span class="text-yellow-500">{{ i ≤ artisan.note ? "★" : "☆" }}</span>
        }
      </div>
      <p class="text-gray-700 mt-2">{{ artisan.location }}</p>
      <div class="flex items-center mt-2">
        
        <p class="text-gray-700">{{ artisan.email }}</p>
      </div>
      <div class="flex items-center mt-2">
        
        <p class="text-gray-700">
          <a
            class="hover:text-blue-500"
            [href]="artisan.website"
            target="_blank"
            >{{ artisan.website }}</a>
        </p>
      </div>
      <p class="text-gray-800 mt-4">{{ artisan.about }}</p>
    </div>
```


Envoi d'Email via EmailJS

Fonctionnement :

- o Le formulaire de contact sur la page de détail d'un artisan permet à l'utilisateur d'envoyer un email à l'artisan.
- o Lorsque l'utilisateur soumet le formulaire, les données du formulaire sont envoyées à EmailJS pour l'envoi de l'email.

Implémentation Technique :

- o EmailJS Service : Utilisation du service EmailjsService pour envoyer les emails.
- o ReactiveFormsModule : Utilisation du module ReactiveFormsModule pour la gestion des formulaires et la validation des champs.
- o Template d'Email : Utilisation d'un template d'email pour personnaliser le contenu de l'email envoyé.

```
// Email sending function with emailjs
onSubmit() {
  if (this.contactForm.valid) {
    const formData = this.contactForm.value;

    const templateParams = {
      from_name: formData.name,
      from_email: formData.email,
      message: formData.message
      // to_email: this.artisan.email // To send to artisan's email
    };

    this.EmailjsService.sendEmail(templateParams).then(
      (response) => {
        alert('Email envoyé avec succès!');
        this.contactForm.reset();
      },
      (error) => {
        alert('Échec de l\'envoi de l\'email.');
```


Template formulaire de contact

```
<!-- Contact form -->
<div class="w-full md:w-1/2 p-4">
  <h2 class="text-xl font-bold mb-4">Contactez-moi</h2>
  <form [formGroup]="contactForm" (ngSubmit)="onSubmit()"
#contactFormRef>
    <div class="mb-4">
      <label class="block text-gray-700 text-sm font-bold mb-2"
for="name">
        Nom
      </label>
      <input
        class="shadow appearance-none border rounded w-full py-2
px-3 text-gray-700 leading-tight focus:outline-none focus:shadow-
outline"
        id="name"
        type="text"
        formControlName="name"
        placeholder="Votre nom"
      />
      <div
        *ngIf="
          contactForm.get('name')?.invalid &&
          contactForm.get('name')?.touched
        "
        class="text-red-500 text-xs italic"
      >
        Le nom est requis.
      </div>
    </div>
    <div class="mb-4">
      <label class="block text-gray-700 text-sm font-bold mb-2"
for="email">
        Email
      </label>
      <input
        class="shadow appearance-none border rounded w-full py-2
px-3 text-gray-700 leading-tight focus:outline-none focus:shadow-
outline"
        id="email"
        type="email"
        formControlName="email"
        placeholder="Votre email"
      />
      <div
        *ngIf="
          contactForm.get('email')?.invalid &&
          contactForm.get('email')?.touched
```

```
"
    class="text-red-500 text-xs italic"
  >
    Veuillez entrer un email valide.
  </div>
</div>
<div class="mb-4">
  <label
    class="block text-gray-700 text-sm font-bold mb-2"
    for="message"
  >
    Message
  </label>
  <textarea
    class="shadow appearance-none border rounded w-full py-2
px-3 text-gray-700 leading-tight focus:outline-none focus:shadow-
outline"
    id="message"
    formControlName="message"
    placeholder="Votre message"
  ></textarea>
  <div
    *ngIf="
      contactForm.get('message')?.invalid &&
      contactForm.get('message')?.touched
    "
    class="text-red-500 text-xs italic"
  >
    Le message est requis.
  </div>
</div>
<div class="flex items-center justify-between">
  <button
    class="bg-blue-500 hover:bg-blue-700 text-white font-bold
py-2 px-4 rounded focus:outline-none focus:shadow-outline cursor-
pointer"
    type="submit"
    [disabled]="!contactForm.valid"
  >
    Envoyer
  </button>
</div>
</form>
</div>
```

Services

Data Service

- o Récupération des données des artisans depuis un fichier JSON.
- o Utilisation de HttpClient pour les requêtes HTTP.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  private dataUrl = 'assets/datas.json'

  constructor(private http: HttpClient) {}

  getData(): Observable<any[]> {
    return this.http.get<any[]>(this.dataUrl);
  }
}
```

Search Service

- o Gestion des résultats de recherche avec un BehaviorSubject pour une mise à jour réactive des résultats.

```
import { Injectable } from '@angular/core';
import { BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class SearchService {
  private searchResults = new BehaviorSubject<any[]>([]);
  searchResults$ = this.searchResults.asObservable();

  updateSearchResults(results: any[]) {
    this.searchResults.next(results);
  }

  constructor() { }
}
```

EmailJS Service

o Intégration d'EmailJS pour l'envoi d'emails depuis le formulaire de contact.

```
import { Injectable } from '@angular/core';
import * as emailjs from 'emailjs-com';

@Injectable({
  providedIn: 'root'
})
export class EmailjsService {

  constructor() { }

  sendEmail(templateParams: any) {
    return emailjs.send('service_xki5kwr', 'template_2zczjol', templateParams, 'QBCL0kZbgk68iHL8Z');
  }
}
```

Responsive Design

- Utilisation de classes utilitaires de Tailwind CSS pour créer un design responsive.
- Adaptation des composants pour différentes tailles d'écran (mobile, tablette, desktop).
- Police utilisée : « Graphik »

Gestion des Routes

- Configuration des routes avec Angular Router pour la navigation entre les différentes pages.
- Utilisation de paramètres de route pour afficher les détails d'un artisan spécifique.
- Utilisation de la méthode '**' pour la gestion des routes inexistantes (page 404)

```
import { Routes } from '@angular/router';
import { HomeComponent } from '../home/home.component';
import { CategoryComponent } from '../category/category.component';
import { ArtisanDetailComponent } from '../artisan-detail/artisan-detail.component';
import { PageNotFoundComponent } from '../page-not-found/page-not-found.component';
import { SearchresultsComponent } from '../searchresults/searchresults.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'artisans/:category', component: CategoryComponent },
  { path: 'artisan/:id', component: ArtisanDetailComponent },
  { path: 'search', component: SearchresultsComponent },
  { path: '**', component: PageNotFoundComponent }
];
```

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Technologies

Gestionnaire de paquets et développer experience

1. NPM
2. Eslint
3. Prettier

Front-end

1. Angular
2. TailwindCSS
3. Angular-router
4. EmailJS
5. GitHub pages
6. VSCode

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre européen de formation*

Chantier, atelier, service ► **Projet de formation**

Période d'exercice ► Du : *Cliquez ici* au : *Cliquez ici*

5. Informations complémentaires (facultatif)

En tant qu'étudiant en développement web, la réalisation de ce projet a été une expérience enrichissante qui m'a permis de mettre en pratique les compétences acquises tout au long de ma formation. Ce projet, développé avec Angular et Tailwind CSS, a non seulement renforcé mes connaissances techniques, mais m'a également confronté à des défis qui ont contribué à mon développement professionnel.

Difficultés Rencontrées

1. *Intégration des Services :*

o L'intégration des services tels que EmailJS pour l'envoi d'emails a été un défi.
Comprendre le fonctionnement des templates d'email et le configurer.

2. Gestion des États avec RxJS :

o La gestion des états avec RxJS, notamment l'utilisation de BehaviorSubject pour les résultats de recherche, a été complexe au début. Comprendre les concepts de programmation réactive et les appliquer de manière efficace a demandé du temps et de la pratique.

3. Validation des Formulaires :

o La validation des formulaires avec ReactiveFormsModule a été un autre défi. Assurer que les champs du formulaire sont correctement validés et que les messages d'erreur sont affichés de manière intuitive.

Savoir-Faire Acquis

1. Maîtrise d'Angular :

o Ce projet m'a permis de renforcer ma maîtrise d'Angular, notamment la création de composants réutilisables, la gestion des routes, et l'utilisation des services pour une architecture modulaire.

2. Utilisation de Tailwind CSS :

o J'ai acquis une solide expérience avec Tailwind CSS, ce qui m'a permis de créer des interfaces utilisateur attrayantes et réactives de manière efficace.

3. Programmation Réactive avec RxJS :

o J'ai développé une meilleure compréhension de la programmation réactive avec RxJS, ce qui m'a permis de gérer les états de manière plus efficace et de créer des applications plus dynamiques.

4. Intégration de Services Externes :

o L'intégration de services externes comme EmailJS m'a permis de comprendre comment interagir avec des API tierces et de les intégrer de manière transparente dans une application Angular.

5. Gestion des Formulaires :

o J'ai appris à créer et valider des formulaires complexes avec ReactiveFormsModule, ce qui est une compétence essentielle pour le développement d'applications web interactives.

Ce projet a été une opportunité précieuse pour appliquer et renforcer mes compétences en développement web. Les défis rencontrés m'ont permis de développer une approche plus méthodique et réfléchie du développement, tout en renforçant ma capacité à résoudre des problèmes complexes. Je suis maintenant mieux préparé à aborder des projets futurs avec confiance et compétence.

Lien repository : <https://github.com/MathP-dev/trouve-ton-artisan>

Activité-type 1

Développer la partie front-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n°3 ► Développer une interface utilisateur web dynamique

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Résumé projet :

J'ai été chargée de développer une interface web dynamique à l'aide du framework Vue.js 3. Dans ce cadre, j'ai conçu un portfolio en ligne destiné à présenter mon profil, mes compétences ainsi que mes premières réalisations.

Ce portfolio répond à plusieurs objectifs. D'une part, il reflète mon identité visuelle et ma personnalité à travers une charte graphique personnalisée. D'autre part, il respecte les contraintes techniques du cahier des charges, notamment l'utilisation de HTML5, CSS3, JavaScript, Vue.js, Git/GitHub ainsi qu'un environnement de développement adapté comme Visual Studio Code.

Le site intègre également plusieurs fonctionnalités essentielles : un header et un footer communs à toutes les pages, une page d'accueil avec une présentation personnelle, une section dédiée aux projets, un formulaire de contact, une fenêtre modale dynamique permettant d'afficher les détails de chaque réalisation, ainsi qu'une page 404 gérée via le routeur.

Au-delà de l'aspect visuel, ce projet prend en compte la sémantique HTML, l'accessibilité, le responsive design, la validité du code selon les recommandations du W3C ainsi qu'une organisation rigoureuse du travail via GitHub. L'objectif était donc de concevoir un portfolio à la fois personnel, fonctionnel et techniquement structuré.

Mise en place du projet

1. La gestion des routes :

La gestion des routes avec VueJs, s'est faite grâce à VueRouter, qui va permettre à l'application de fonctionner comme une SPA (Single Page Application).

Cela permet un accès facile à l'utilisateur à la page d'accueil, des projets et au formulaire de contact. Une page 404 a été mise en place pour gérer les URL inconnues ou incorrectes.

Router (router/index.js)

```
1 import { createRouter, createWebHistory } from "vue-router";
2 import Home from "../views/Home.vue";
3 import NotFound from "../views/NotFound.vue";
4
5 const router = createRouter({
6   history: createWebHistory(import.meta.env.BASE_URL),
7   routes: [
8     {
9       path: "/",
10      name: "home",
11      component: Home,
12    },
13    {
14      path: "/*",
15      name: "not-found",
16      component: NotFound,
17    },
18  ],
19   scrollBehavior(to, from, savedPosition) {
20     if (to.hash) {
21       return {
22         el: to.hash,
23         behavior: "smooth",
24       };
25     }
26     return { top: 0 };
27   },
28 });
29
30 export default router;
```


Router injecté dans App.vue

```
1  <template>
2    <div id="app">
3      <BurgerMenu />
4      <Header />
5      <router-view></router-view>
6      <Footer />
7    </div>
8  </template>
9
10 <script>
11 import Header from "../components/Header.vue";
12 import Footer from "../components/Footer.vue";
13 import BurgerMenu from "../components/BurgerMenu.vue";
14
15 export default {
16   name: "App",
17   components: {
18     Header,
19     Footer,
20     BurgerMenu,
21   },
22 };
23 </script>
24
25 <style>
26 #app {
27   text-align: center;
28   font-family: "Candal", sans-serif;
29   -webkit-font-smoothing: antialiased;
30   -moz-osx-font-smoothing: grayscale;
31   color: #2c3e50;
32   background-image: url("@/assets/sandra-ilic_lgZwvaxGVg-unsplash.jpg");
33   background-size: cover;
34   background-repeat: no-repeat;
35   background-position: center;
36   background-attachment: fixed;
37 }
38 </style>
39
```

2. Modale et carrousel dynamique

La fenêtre modale dynamique est une fonctionnalité importante du projet. Les projets sont stockés sous forme d'objets contenant plusieurs informations : image, titre, date, description, technologies utilisées, ainsi que deux boutons permettant de télécharger un document PDF ou d'accéder au repository GitHub.

J'ai également développé le fait de faire tourner ces modals en 'carrousel', ce qui rajoute du dynamisme et de la fluidité sur la page.

Template modal :

```
<template>
  <div class="modal" @click.self="$emit('close')">
    <div class="modal-content">
      <button @click="$emit('close')" class="close-button">&times;</button>
      <h2>{{ project.title }}</h2>
      <p>Date de création : {{ project.date }}</p>
      <p>Technologies : {{ project.technologies.join(", ") }}</p>

      <!-- Images supplémentaires -->
      <div class="additional-images">
        <h3>Images supplémentaires :</h3>
        <div class="image-gallery">
          <a
            v-for="(img, index) in project.additionalImages"
            :key="img- + index"
            :href="img"
            target="_blank"
          >
            
          </a>
        </div>
      </div>
      <a :href="project.link" target="_blank" class="project-link">
        >Visiter le projet</a>
      <br />
      <a
        v-if="project.github && project.title !== 'Cahier des charges'"
        :href="project.github"
        target="_blank"
        class="github-link"
      >
        >Voir sur GitHub</a>
      </div>
    </div>
  </div>
</template>
```

Script :

```
const selectedProject = ref(null);
const showModal = ref(false);

const openModal = (project) => {
  selectedProject.value = project;
  showModal.value = true;
};

const closeModal = () => {
  showModal.value = false;
};
```

```
<script>
export default {
  name: "ProjectModal",
  props: ["project"],
};
</script>
```

```
<ProjectModal
  :project="selectedProject"
  v-if="showModal"
  @close="closeModal"
/>
```

CSS :

```
<style scoped>
.modal {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  display: flex;
  justify-content: center;
  align-items: center;
}

.modal-content {
  background-color: rgba(8 51 68);
  padding: 2rem;
  border-radius: 8px;
  position: absolute;
}

.close-button {
  position: absolute;
  top: 10px;
  right: 10px;
  font-size: 1.5rem;
  background: none;
  border: none;
  cursor: pointer;
}

.main-image {
  width: 100%;
  max-height: 300px;
}

.additional-images {
  margin-top: 20px;
}

.image-gallery {
  display: flex;
}
```

Carrousel des modals :

Le carrousel est un ajout qui permet de dynamiser l'affichage des modals.

Template :

```
<section id="projects">
  <h2>Mes Projets</h2>
  <div class="carousel">
    <div class="carousel-inner" :style="{ width: `${totalWidth}px` }">
      <!-- Dupliquer les projets pour l'effet de continuité -->
      <div
        v-for="project in duplicatedProjects"
        :key="project.id"
        class="project-card"
        @click="openModal(project)"
      >
        
        <h3>{{ project.title }}</h3>
      </div>
    </div>
  </div>
</section>
```

Script :

```
// Dupliquer les projets pour l'effet de continuité
const duplicatedProjects = [...projects, ...projects];

// Calculer la largeur totale du carrousel
const totalWidth = computed(() => duplicatedProjects.length * 300);
```

CSS :

```
<style scoped>
.carousel {
  overflow: hidden; /* Masquer le débordement */
}

.carousel-inner {
  display: flex; /* Pour permettre le défilement horizontal */
  animation: scroll linear infinite; /* Animation continue */
  animation-duration: 15s; /* Durée d'animation dynamique */
}

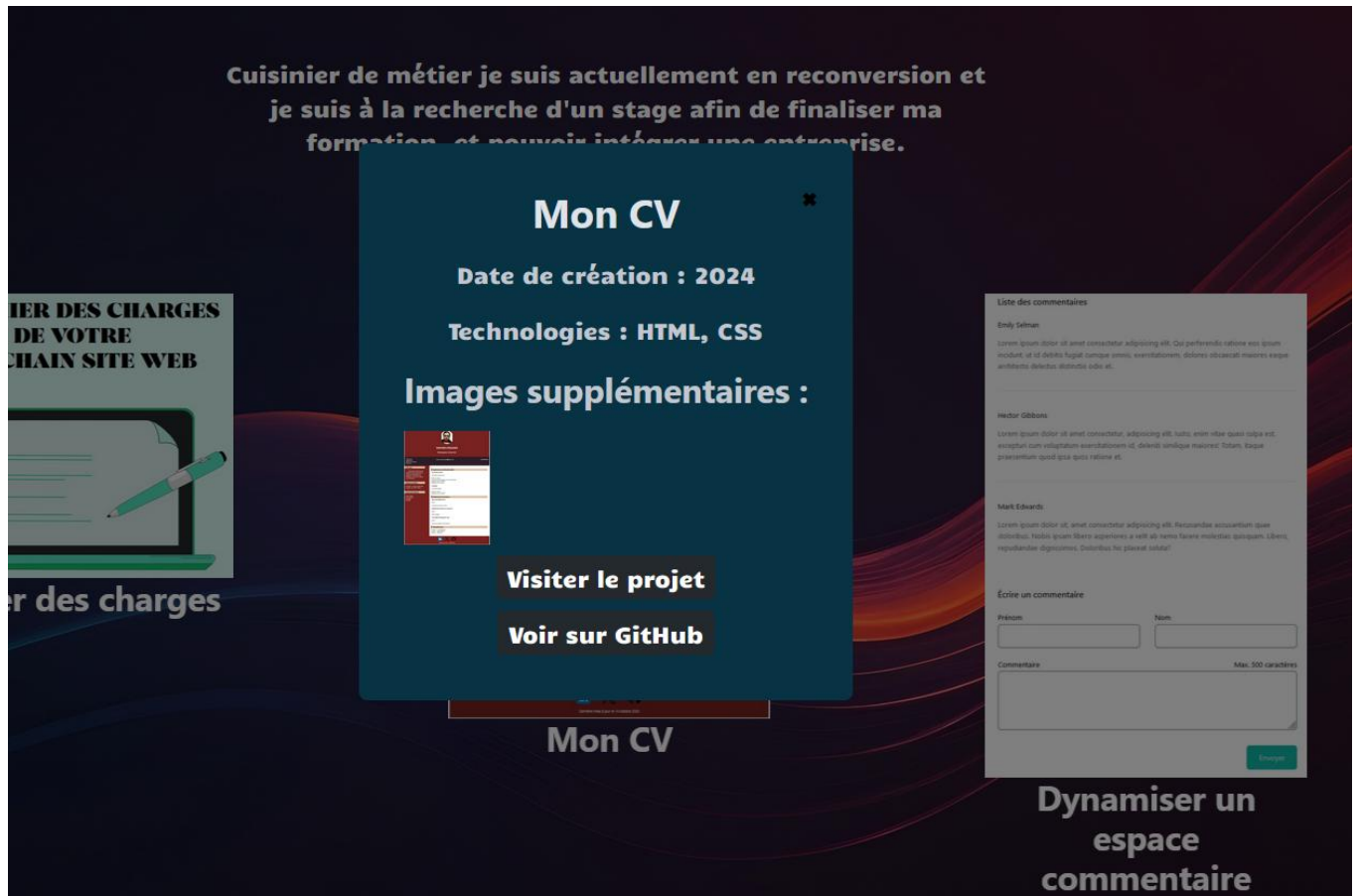
@keyframes scroll {
  0% {
    transform: translateX(0); /* Commence au début */
  }

  100% {
    transform: translateX(
      -50%
    ); /* Fin à gauche (50% pour deux ensembles de projets) */
  }
}

.project-card {
  flex-shrink: 0; /* Empêche le rétrécissement des cartes */
  width: 300px;
  margin-right: 200px; /*espacement entre les cartes */
}

.project-image {
  width: 100%;
  height: auto;
}
```

Interface du portfolio illustrant l'ouverture de la fenêtre modale affichant les détails d'un projet.



3. Formulaire de contact

Le formulaire de contact est une autre partie dynamique du site où les données sont liées au V-model de VueJS. Lorsque le formulaire est soumis, les données sont récupérée et stockée, puis partagée avec le service externe EmailJS.

Template :

```
32 <section id="contact">
33   <h2>Contact</h2>
34   <form @submit.prevent="submitForm" class="contact-form">
35     <div>
36       <label for="name">Nom/Prénom :</label>
37       <input type="text" id="name" v-model="nameForm" required />
38     </div>
39     <div>
40       <label for="subject">Objet :</label>
41       <input type="text" id="subject" v-model="subject" required />
42     </div>
43     <div>
44       <label for="message">Message :</label>
45       <textarea id="message" v-model="message" required></textarea>
46     </div>
47     <button type="submit">Envoyer</button>
48   </form>
49 </section>
```

DOSSIER PROFESSIONNEL (DP)

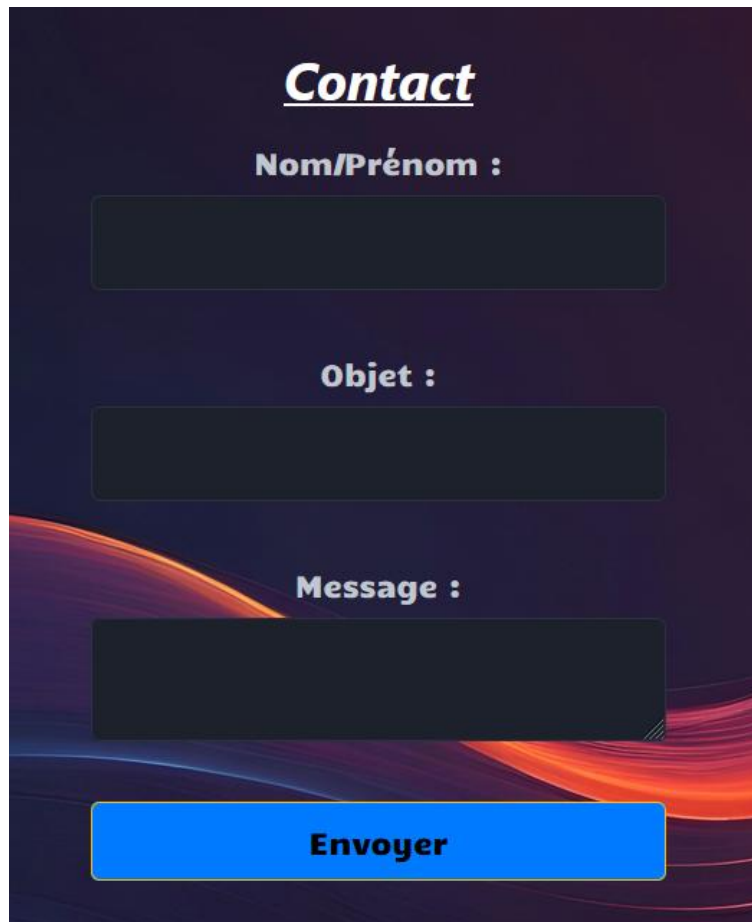
Script :

```
62 <section id="contact">
63   <h2>Contact</h2>
64   <form @submit.prevent="submitForm" class="contact-form">
65     <div>
66       <label for="name">Nom/Prénom :</label>
67       <input type="text" id="name" v-model="nameForm" required />
68     </div>
69     <div>
70       <label for="subject">Objet :</label>
71       <input type="text" id="subject" v-model="subject" required />
72     </div>
73     <div>
74       <label for="message">Message :</label>
75       <textarea id="message" v-model="message" required></textarea>
76     </div>
77     <button type="submit">Envoyer</button>
78   </form>
79 </section>
```

Variables d'environnement :

```
1 VITE_CONTACT_EMAIL=mathieu.paquier85@gmail.com
2 VITE_EMAILJS_SERVICE_ID=service_cgn628c
3 VITE_EMAILJS_TEMPLATE_ID=template_2zczjol
4 VITE_EMAILJS_USER_ID=QBCL0kZbgk68iHL8Z
```

Vue :



Contact

Nom/Prénom :

Objet :

Message :

Envoyer

4. Création d'un menu 'burger'

J'ai aussi mis en place un menu 'burger' car ils sont devenus essentiels dans le web moderne, principalement pour l'adaptabilité mobile. Avec la majorité du trafic web provenant désormais de smartphones et tablettes, l'espace d'écran est limité. Le menu burger permet de masquer la navigation dans une barre compacte pour ne pas encombrer l'interface, tout en la rendant accessible d'un simple clic.

Sur les versions desktop, même avec plus d'espace, ce type de menu offre une meilleure **expérience utilisateur** en réduisant la surcharge visuelle et en organisant le contenu de façon hiérarchisée.

Un menu burger bien implémenté favorise une **meilleure navigation** et peut réduire le taux de rebond en gardant le design épuré et professionnel.

Template :

```
<template>
  <div>
    <div class="burger-menu" :class="{ active: isOpen }">
      <button @click="toggleMenu" class="burger-button">
        <span class="burger-bar"></span>
        <span class="burger-bar"></span>
        <span class="burger-bar"></span>
      </button>
      <nav class="menu-items" @click.stop>
        <ul>
          <li><router-link to="/">Accueil</router-link></li>
          <li><router-link to="/#about">À propos</router-link></li>
          <li><router-link to="/#projects">Mes projets</router-link></li>
          <li><router-link to="/#contact">Contact</router-link></li>
        </ul>
      </nav>
    </div>
    <div v-if="isOpen" class="overlay" @click="closeMenu"></div>
  </div>
</template>
```

Script :

```
<script setup>
import { ref, watch, onMounted, onUnmounted } from "vue";
import { useRouter } from "vue-router";

const router = useRouter();
const isOpen = ref(false);

const toggleMenu = () => {
  isOpen.value = !isOpen.value;
};

const closeMenu = () => {
  isOpen.value = false;
};

// Fermer le menu lorsque la route change
watch(
  () => router.currentRoute.value,
  () => {
    closeMenu();
  }
);

const handleOutsideClick = (event) => {
  if (isOpen.value && !event.target.closest(".burger-menu")) {
    closeMenu();
  }
};

onMounted(() => {
  document.addEventListener("click", handleOutsideClick);
});

onUnmounted(() => {
  document.removeEventListener("click", handleOutsideClick);
});
</script>
```

DOSSIER PROFESSIONNEL (DP)

CSS :

```
<style scoped>
.burger-menu {
  position: fixed;
  top: 20px;
  left: 20px;
  z-index: 1001;
}

.burger-button {
  position: relative;
  display: flex;
  flex-direction: column;
  justify-content: space-around;
  width: 30px;
  height: 25px;
  background: transparent;
  border: none;
  cursor: pointer;
  padding: 0;
  z-index: 1002;
}

.burger-bar {
  width: 30px;
  height: 3px;
  background-color: white;
  transition: all 0.3s linear;
}

.active .burger-bar:nth-child(1) {
  transform: rotate(45deg) translate(5px, 5px);
}

.active .burger-bar:nth-child(2) {
  opacity: 0;
}

.active .burger-bar:nth-child(3) {
  transform: rotate(-45deg) translate(7px, -6px);
}

.overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  z-index: 1000;
}

nav {
  border-radius: 0 0 40% 5%;
}
</style>

.menu-items {
  position: fixed;
  top: 0;
  left: -100%;
  background-color: #642b2b;
  transition: left 0.3s ease-in-out;
  z-index: 1000;
  padding: 20px;
  box-shadow: 2px 0 5px rgba(0, 0, 0, 0.1);
}

.active .menu-items {
  left: 0;
}

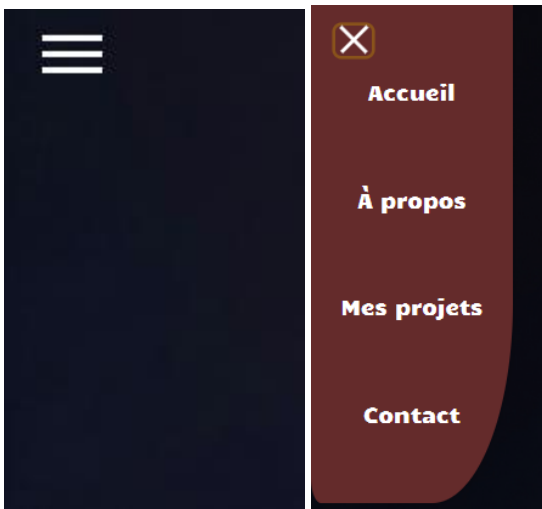
ul {
  list-style-type: none;
  padding: 0;
  margin: 0;
  margin-top: 20px;
  display: flex;
  flex-direction: column;
}

li {
  margin-bottom: 15px;
}

a {
  text-decoration: none;
  color: #ffffff;
  font-size: 18px;
  display: block;
  padding: 10px 15px;
  transition: background-color 0.3s;
}

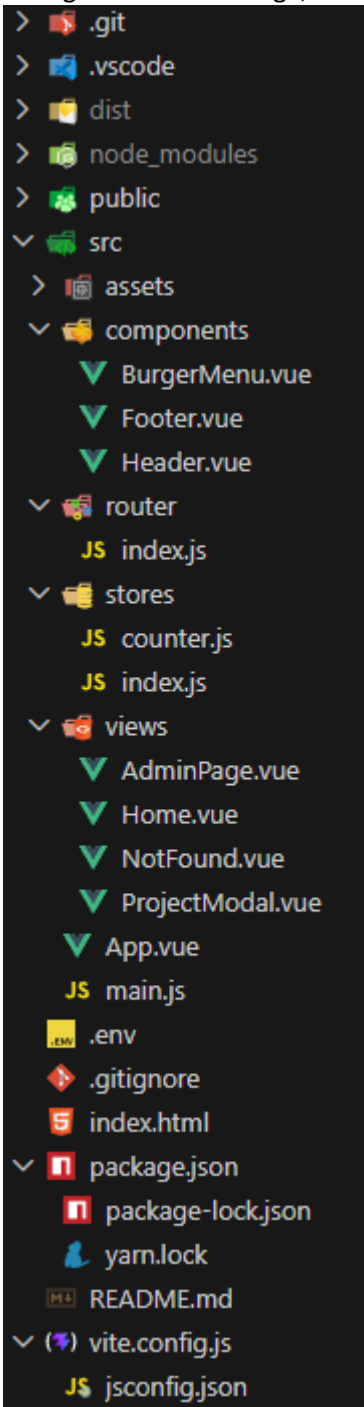
a:hover {
  background-color: #726c6c;
}
```

Vue :



5. Organisation du projet :

Un projet Vue.js suit généralement une structure organisée et modulaire pour faciliter la maintenance et la scalabilité. Le dossier **src/** contient les fichiers source, avec notamment un dossier **components/** regroupant les composants réutilisables, un dossier **views/** pour les pages principales (souvent liées au routage), et un dossier **assets/** contenant images, styles et autres ressources statiques. On retrouve aussi un dossier **router/** pour la configuration du routage, un dossier **stores/** pour la gestion centralisée de l'état de l'application.



DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

- Technologies : HTML, CSS, JavaScript, Vue.js, Vite, Vue Router, EmailJS, Yarn
- Méthodes de mise en page : CSS3, Flexbox
- Outils de développement : Visual Studio Code, Git, GitHub

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre européen de formation*

Chantier, atelier, service ► **Projet de formation**

Période d'exercice ► Du : [Cliquez ici](#) au : [Cliquez ici](#)

5. Informations complémentaires (facultatif)

Bilan :

Je suis ravi d'avoir créé mon portfolio en utilisant Vue.js, qui reflète ma maîtrise des bases du développement web et ma capacité à travailler avec un framework moderne. Ce site sert de vitrine pour mes compétences récemment acquises et présente mes premières réalisations de manière créative et personnalisée. Dans mon portfolio, j'ai inclus plusieurs sections clés : une page d'accueil où je me présente, une section "À propos" qui détaille mon parcours et mes motivations, ainsi qu'un espace dédié à mes projets, comprenant mon CV, le cahier des charges et un site communautaire que j'ai précédemment réalisé. J'ai également mis en place une galerie de compétences pour mettre en avant mes connaissances techniques, ainsi qu'un formulaire de contact pour les employeurs potentiels. J'ai veillé à respecter l'ergonomie, l'accessibilité et la sémantique de mon site, en m'assurant que le code HTML et CSS soit valide selon les normes du W3C. Cela témoigne de mon souci du détail et de mon professionnalisme. Grâce à Vue.js, j'ai pu créer une interface utilisateur réactive et dynamique, offrant une expérience fluide aux visiteurs. J'ai intégré des fonctionnalités telles que le routage pour une navigation sans rechargement et utilisé Pinia pour gérer les données dynamiques. Ce portfolio est ma carte de visite pour ma future carrière de développeur, montrant ma capacité à concevoir et réaliser un projet web complet et fonctionnel.

Lien Github du projet : <https://github.com/MathP-dev/ProjetPortfolio>

Activité-type 3

Développer la partie front-end d'une application web ou mobile en intégrant les recommandations de sécurité

Exemple n° 4 ► Réaliser une interface utilisateur avec une solution de gestion de contenu ou e-commerce

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Résumé du projet :

La société La vie des plantes est une entreprise qui vend des kits de plantes à faire pousser, des graines, etc.

Elle souhaite mettre en place un site internet, avec une partie e-commerce pour pouvoir diversifier ses clients et augmenter son chiffre d'affaires.

Elle m'a contacté pour créer son site e-commerce :

- partir d'une nouvelle installation de WordPress afin de créer le site internet appelé La_vie_des_plantes ;
- utiliser le thème Flower Shop Lite choisi par La vie des plantes
- le paramétrer afin qu'il respecte la charte graphique
- intégrer les prix, photos et descriptions des trois produits à mettre en vente.

La page d'accueil est composée d'une courte description, de l'affichage des trois articles avec des boutons pour ajouter au panier, ou pour aller à la fiche produit.

L'entreprise cible une population jeune, entre 25 et 35 ans, urbaine, ce qui m'a amené à privilégier une interface simple, épurée, moderne et visuellement végétale.

Configuration du projet :

Le projet a été réalisé à partir d'une nouvelle installation de WordPress.

Modules installés : WooCommerce, Elementor, Flower Shop Lite

Ce qui a été configuré pour le site :

- Nom du site
- Menu
- Contenus des différentes pages
- Boutique et panier
- Couleur, logo, favicon

Charte graphique :

Polices :

- Noto Sans pour les titres
- Chakra Petch pour les textes courant

DOSSIER PROFESSIONNEL (DP)

Logo :



Favicon :



Palette de couleur :



Réalisation du site :

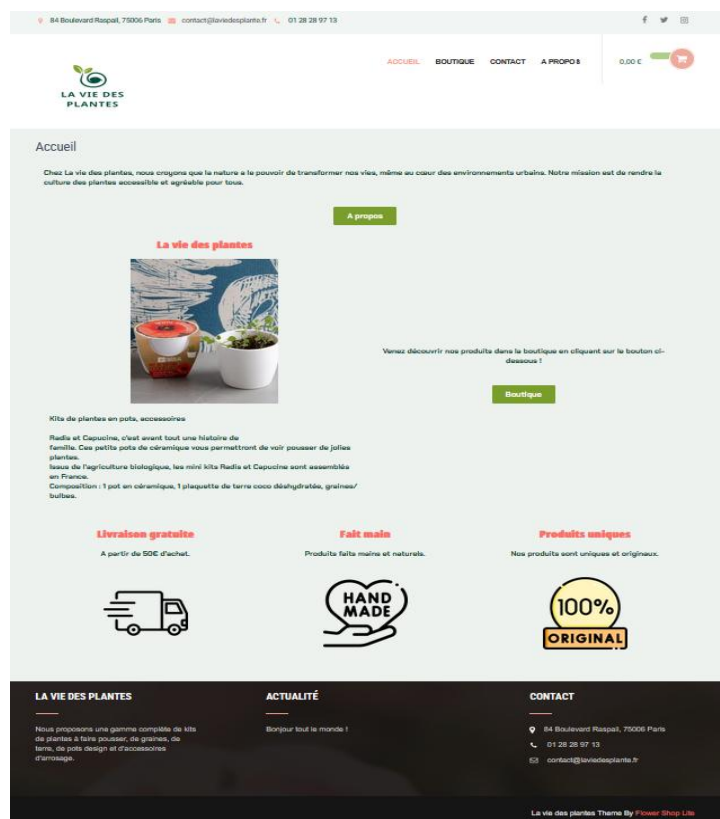
Les différentes pages :

- Accueil
- Contact
- À propos
- WooCommerce : Panier, Commande, produits

Un header et footer sont présent sur toutes les pages.

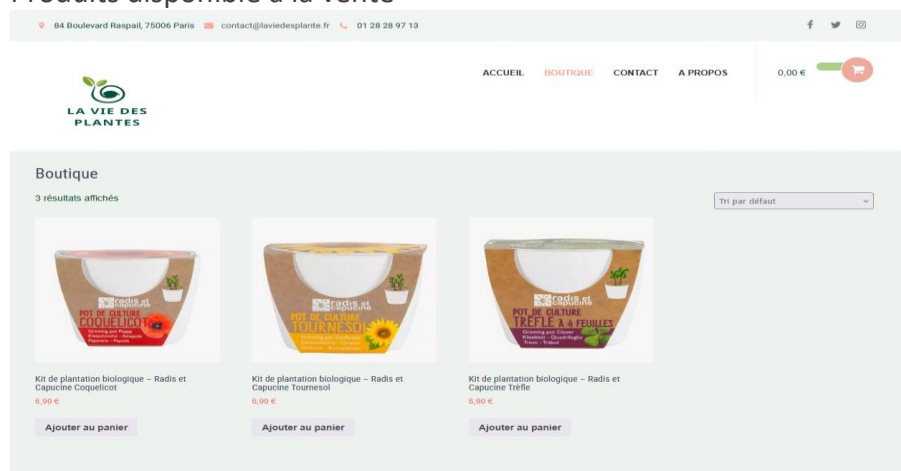
Page Accueil :

- une présentation de l'entreprise,
- un bouton de redirection vers la page "À propos",
- une section "Nos produits" avec les trois articles,
- des blocs informatifs (livraison, fait main, produits uniques).



Page boutique :

Produits disponible à la vente



DOSSIER PROFESSIONNEL (DP)

Fiche d'un produit :

Détail d'un produit avec visuel, prix, description, composition et produit similaire.

84 Boulevard Raspail, 75006 Paris | contact@laviedesplantes.fr | 01 28 28 97 13

ACCUEIL BOUTIQUE CONTACT A PROPOS 0,00 €

LA VIE DES PLANTES

Kit de plantation biologique – Radis et Capucine Coquelicot

6,90 €

Ajouter au panier

Catégorie : Non classé

Description

Radis et Capucine : Radis et Capucine, c'est avant tout une histoire de famille. Ces petits pots de céramique vous permettront de voir pousser de jolies plantes, issues de l'agriculture biologique, les mini kits Radis et Capucine sont assemblés en France.

Composition : 1 pot en céramique, 1 plaquette de terre oséo déshydratée, graines/bulbes.

Produits similaires

Page commande/panier :

84 Boulevard Raspail, 75006 Paris | contact@laviedesplantes.fr | 01 28 28 97 13

ACCUEIL BOUTIQUE CONTACT A PROPOS 6,90 € 1

LA VIE DES PLANTES

Panier

PRODUIT	TOTAL	TOTAL PANIER
 Kit de plantation biologique – Radis et Capucine Coquelicot 6,90 € Radis et Capucine... - 1 + Supprimer l'élément	6,90 €	Ajouter un code promo Sous-total 6,90 € Total 6,90 €

[Valider la commande](#)

LA VIE DES PLANTES

Nous proposons une gamme complète de kits de plantes à faire pousser, de graines, de terre, de pots design et d'accessoires d'arrosage.

ACTUALITÉ

Bonjour tout le monde !

CONTACT

84 Boulevard Raspail, 75006 Paris
01 28 28 97 13
contact@laviedesplantes.fr

DOSSIER PROFESSIONNEL (DP)

Validation de commande :

84 Boulevard Raspail, 75006 Paris | contact@laviedesplantes.fr | 01 28 28 97 13

f t i



ACCUEIL BOUTIQUE CONTACT A PROPOS

6,90 € 1

Validation de la commande

Coordonnées
Nous utiliserons cet e-mail pour vous envoyer des détails et des mises à jour concernant votre commande.

Adresse e-mail
mathieu.paquier85@gmail.com

Adresse de facturation
Entrez l'adresse de facturation qui correspond à votre moyen de paiement.

Pays/Région
France

Prénom Nom

Adresse

+ Ajouter appartement, suite, etc.

Code postal Ville

Numéro de téléphone (facultatif)

Options de paiement

Résumé de la commande

1	Kit de plantation biologique – Radis et Capucine Coquelicot	6,90 €
	Radis et Capucine...	
Ajouter un code promo		▼
Sous-total		6,90 €
Total		6,90 €

Page à propos :

Identité de l'entreprise, histoire, photo de l'équipe.

84 Boulevard Raspail, 75006 Paris | contact@laviedesplantes.fr | 01 28 28 97 13

f t i



ACCUEIL BOUTIQUE CONTACT A PROPOS

6,90 € 1

A propos

Bienvenue chez La vie des plantes !

Chez La vie des plantes, nous croyons que la nature a le pouvoir de transformer nos vies, même au cœur des environnements urbains. Notre mission est de rendre la culture des plantes accessible et agréable pour tous.

Qui Sommes-Nous ?

Fondée par des passionnés de botanique et de design, La vie des plantes est bien plus qu'une simple boutique en ligne. Nous sommes une communauté dédiée à l'art de faire pousser des plantes, à la découverte de nouvelles espèces et à la création d'espaces verts inspirants, même dans les plus petits appartements.

Nos Produits : Nous proposons une gamme complète de kits de plantes à faire pousser, de graines, de terre, de pots design et d'accessoires d'arrosage. Chaque produit est soigneusement sélectionné pour sa qualité et son esthétique, afin de s'intégrer parfaitement dans votre intérieur moderne.

- **Kits de Plantes :** Tout ce dont vous avez besoin pour démarrer, avec des instructions simples et des graines de qualité.
- **Graines :** Une variété de graines pour tous les niveaux d'expérience, des plantes faciles d'entretien aux espèces plus exotiques.
- **Terre et Substrats :** Des mélanges spécialement formulés pour assurer une croissance optimale de vos plantes.
- **Pots et Accessoires :** Des designs élégants et fonctionnels pour sublimer vos plantes et faciliter leur entretien.



Notre Engagement : Nous nous engageons à promouvoir des pratiques durables et respectueuses de l'environnement. Nos emballages sont recyclables, et nous privilégions les fournisseurs locaux pour réduire notre empreinte carbone.

Rejoignez-Nous ! Que vous soyez un jardinier en herbe ou un passionné confirmé, La vie des plantes est là pour vous accompagner dans votre aventure botanique. Suivez-nous sur les réseaux sociaux pour des conseils, des astuces et des inspirations, et rejoignez notre communauté de plant lovers !

LA VIE DES PLANTES

ACTUALITÉ

CONTACT

DOSSIER PROFESSIONNEL (DP)

Page contact :

Pour la prise de contact avec l'entreprise grâce à un formulaire, ajout d'une carte pour la localisation de l'entreprise.

84 Boulevard Raspail, 75006 Paris contact@laviedesplante.fr 01 28 28 97 13

f t i

ACCUEIL BOUTIQUE **CONTACT** A PROPOS 6,90 € 1

LA VIE DES PLANTES

Contact

Contactez-nous

Nom :

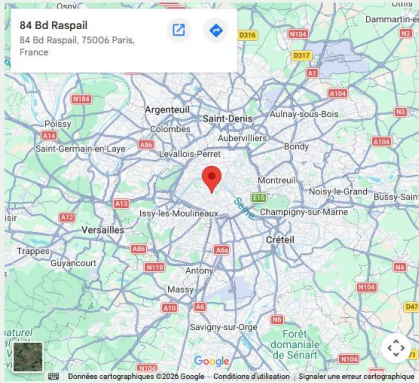
Email :

Sujet :

Message :

Envoyer

84 Bd Raspail
84 Bd Raspail, 75006 Paris, France



Header et Footer :

Ils sont présents sur toutes les pages.

Header :

84 Boulevard Raspail, 75006 Paris contact@laviedesplante.fr 01 28 28 97 13

f t i

LA VIE DES PLANTES

ACCUEIL BOUTIQUE **CONTACT** A PROPOS 6,90 € 1

Footer :

LA VIE DES PLANTES

Nous proposons une gamme complète de kits de plantes à faire pousser, de graines, de terre, de pots design et d'accessoires d'arrosage.

ACTUALITÉ

Bonjour tout le monde !

CONTACT

84 Boulevard Raspail, 75006 Paris

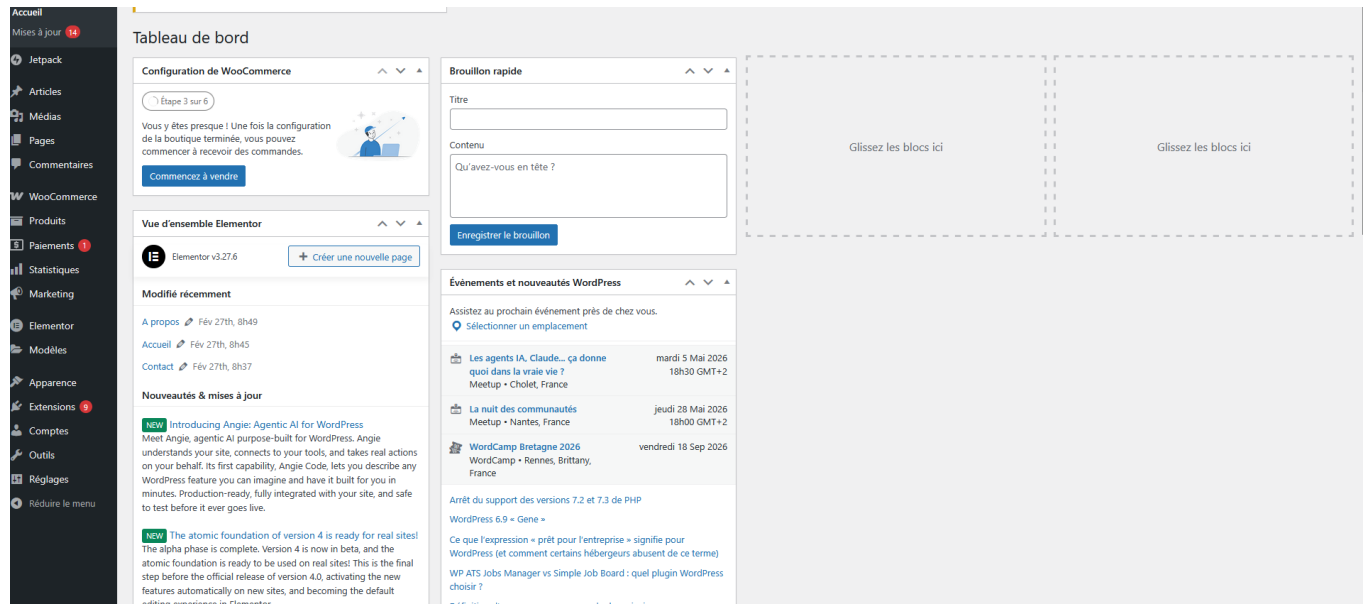
01 28 28 97 13

contact@laviedesplante.fr

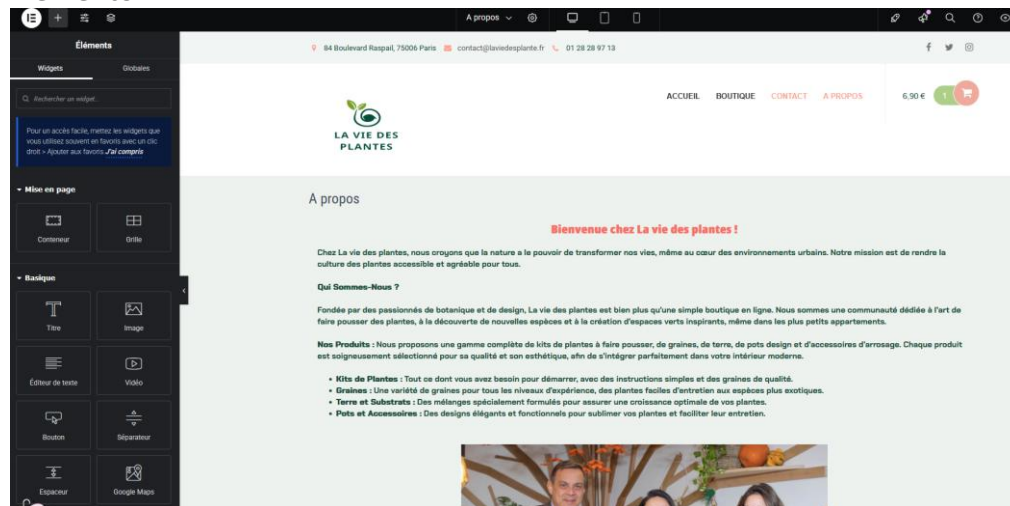
DOSSIER PROFESSIONNEL (DP)

Tableau de bord WordPress :

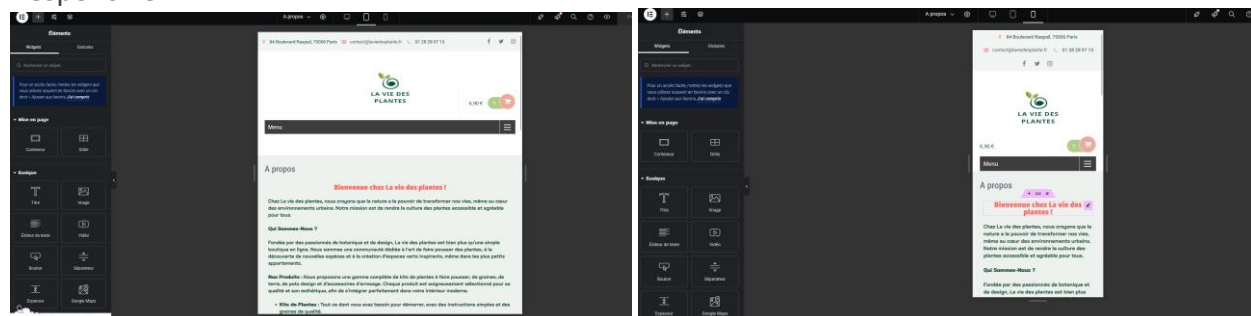
- Gestion des pages, extensions, contenus
- Elementor pour le visuel et responsive
- WooCommerce pour la gestion produits/ventes



Elementor :



Responsive :



DOSSIER PROFESSIONNEL (DP)

Gestion produits :

☐	Nom	UGS	Stock	Prix	Catégories	Étiquettes	Brands	Date
☐	Kit de plantation biologique – Radis et Capucine Trèfle	–	En stock	6,90 €	Non classé	–	–	Publié 25/02/2025 à 8h05
☐	Kit de plantation biologique – Radis et Capucine Coquelicot	–	En stock	6,90 €	Non classé	–	–	Publié 25/02/2025 à 8h05
☐	Kit de plantation biologique – Radis et Capucine Tournesol	–	En stock	6,90 €	Non classé	–	–	Publié 25/02/2025 à 7h38

2. Précisez les moyens utilisés :

Technologies: WordPress (CMS), WooCommerce (e-commerce), HTML / CSS, PHP (via WordPress), MySQL / MariaDB.

Méthodes de mise en page : Thème Flower Shop Lite, Construction des pages avec Elementor

Outils de développement : WordPress, Elementor, WooCommerce, GitHub, WAMP / AlwaysData, phpMyAdmin

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre européen de formation*

Chantier, atelier, service ► *Projet de formation*

Période d'exercice ► Du : *Cliquez ici* au : *Cliquez ici*

5. Informations complémentaires *(facultatif)*

Bilan :

En développant mon propre projet WordPress, j'ai découvert une plateforme extrêmement puissante et accessible, même pour quelqu'un qui débute. WordPress m'a permis de créer rapidement un site fonctionnel sans avoir besoin de coder from scratch, grâce à ses thèmes et plugins prêts à l'emploi.

J'ai appris comment fonctionne vraiment une plateforme CMS (Content Management System), comment gérer le contenu, les utilisateurs et les permissions de manière centralisée. En creusant davantage, j'ai compris la structure des bases de données WordPress, comment les thèmes et plugins s'intègrent dans l'écosystème, et comment personnaliser le tout avec du PHP quand nécessaire.

J'ai aussi découvert l'importance de la sécurité (mises à jour, plugins de sécurité, sauvegardes), de l'optimisation des performances, et du SEO natif de WordPress. Ce projet m'a vraiment montré la différence entre développer un site from scratch et utiliser une solution établie : flexibilité vs rapidité, contrôle total vs facilité d'utilisation.

Lien repository Github : https://github.com/MathP-dev/la_vie_des_plantes

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n° 1 ► Créer une base de données

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

1. Résumé de la mission

La société Knowledge, spécialisée dans l'édition de livres de formation souhaite développer son offre de formation, en proposant une plateforme e-learning à ses clients, sur laquelle ils pourront étudier de chez eux et en toute autonomie.

J'ai donc été mandaté pour concevoir le back-end de son site e-learning/e-commerce.

Ma mission sera donc de concevoir une base de données adaptée, des composants d'accès aux données ainsi que les composants e-commerce pour répondre au périmètre fonctionnel du projet « Knowledge Learning ».

Périmètre fonctionnel du projet

Voici la liste des fonctionnalités souhaitées par l'éditeur pour son site :

- L'utilisateur peut créer un compte qu'il devra activer par mail.
- L'utilisateur peut se connecter à son compte.
- Un utilisateur qui n'a pas activé son compte ne peut acheter de cursus ou de leçon.
- L'utilisateur ayant un rôle administrateur pourra accéder au backoffice pour gérer les comptes utilisateurs, les contenus, les achats.
- L'utilisateur ayant un rôle client pourra acheter un cursus entier ou une leçon spécifique.
- Acheter une leçon ou un cursus signifie obtenir les droits d'accès aux pages des leçons concernées par l'achat.
- L'utilisateur ayant un rôle client pourra obtenir une certification « Knowledge Learning » s'il a validé toutes les leçons d'un thème.

Hierarchie métier :

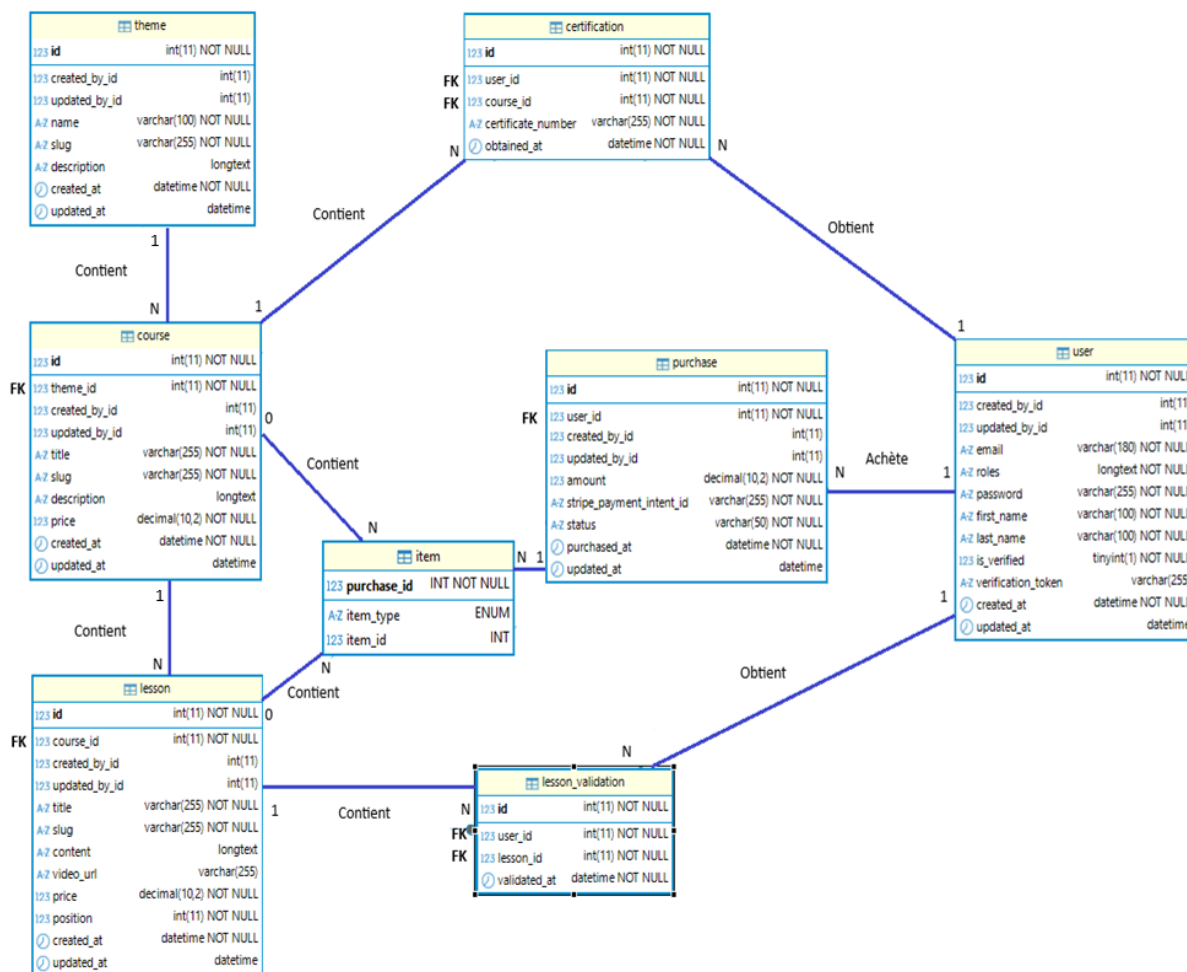
THEME (1)->(N) COURSE (1)->(N) LESSON

2. Conception de la base de données

L'accès aux données est assuré par l'ORM Doctrine, qui permet de manipuler les entités de l'application et d'interagir avec la base de données sans écrire directement de requêtes SQL. Doctrine facilite la gestion des relations entre les différentes entités et simplifie les opérations de lecture, création, modification et suppression des données.

La base de données est structurée de façon hiérarchique : un Theme regroupe un ou plusieurs Course (cursus), et chaque Course contient un ou plusieurs Lesson (leçons). Un User peut acheter un cursus et/ou une leçon via une Purchase et Item, puis valider sa progression au fil des LessonValidation. Une fois les leçons d'un cursus complétées, l'utilisateur obtient une Certification associée au cursus.

Modélisation de la base de données :



DOSSIER PROFESSIONNEL (DP)

Table User

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	11	PRIMARY KEY, NOT NULL	-	Identifiant unique de l'utilisateur
created_by_id	INT	11	NOT NULL	-	ID de l'utilisateur créateur
updated_by_id	INT	11	NOT NULL	-	ID de l'utilisateur ayant modifié
email	VARCHAR	180	NOT NULL, UNIQUE	-	Adresse email de l'utilisateur
password	VARCHAR	255	NOT NULL	-	Mot de passe haché
first_name	VARCHAR	100	NOT NULL	-	Prénom de l'utilisateur
last_name	VARCHAR	100	NOT NULL	-	Nom de famille de l'utilisateur
is_verified	TINYINT	1	NOT NULL	-	Indicateur de vérification du compte
verification_token	VARCHAR	255	-	-	Code de vérification d'email
created_at	DATETIME	-	NOT NULL	-	Date/heure de création
updated_at	DATETIME	-	NOT NULL	-	Date/heure de dernière modification

Contraintes métier :

- L'inscription et l'authentification,
- La vérification du compte par mail,
- Le lien avec les achats, validations et certificats.

DOSSIER PROFESSIONNEL (DP)

Table Theme

Cette table représente les grands domaines de formation proposés sur la plateforme.

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	11	PRIMARY KEY, NOT NULL	-	Identifiant unique du thème
created_by_id	INT	11	NOT NULL	-	ID de l'utilisateur qui a créé le thème
updated_by_id	INT	11	NOT NULL	-	ID de l'utilisateur qui a modifié le thème
name	VARCHAR	100	NOT NULL	-	Nom du thème
slug	VARCHAR	255	NOT NULL	-	Slug URL du thème
description	LONGTEXT	-	-	-	Description complète du thème
created_at	DATETIME	-	NOT NULL	-	Date/heure de création
updated_at	DATETIME	-	NOT NULL	-	Date/heure de dernière modification

Plus haut niveau de l'arborescence, regroupe 1 ou plusieurs cursus.

Table Course

Cette table représente un parcours de formation rattaché à un thème

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	11	PRIMARY KEY, NOT NULL	-	Identifiant unique du cours
theme_id	INT	11	FOREIGN KEY, NOT NULL	-	Référence à la table theme
created_by_id	INT	11	NOT NULL	-	ID de l'utilisateur créateur
updated_by_id	INT	11	NOT NULL	-	ID de l'utilisateur ayant modifié
title	VARCHAR	255	NOT NULL	-	Titre du cours
slug	VARCHAR	255	NOT NULL	-	Slug URL du cours
description	LONGTEXT	-	-	-	Description détaillée du cours
price	DECIMAL	10,2	NOT NULL	-	Prix du cours
created_at	DATETIME	-	NOT NULL	-	Date/heure de création
updated_at	DATETIME	-	NOT NULL	-	Date/heure de dernière modification

Clés étrangères : theme_id → theme(id)

Appartient à Theme et contient 1 ou plusieurs leçon(s).

DOSSIER PROFESSIONNEL (DP)

Table Lesson

Cette table représente une leçon appartenant à un cursus.

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	11	PRIMARY KEY, NOT NULL	-	Identifiant unique de la leçon
course_id	INT	11	FOREIGN KEY, NOT NULL	-	Référence à la table course
created_by_id	INT	11	NOT NULL	-	ID de l'utilisateur créateur
updated_by_id	INT	11	NOT NULL	-	ID de l'utilisateur ayant modifié
title	VARCHAR	255	NOT NULL	-	Titre de la leçon
slug	VARCHAR	255	NOT NULL	-	Slug URL de la leçon
content	LONGTEXT	-	NOT NULL	-	Contenu complet de la leçon
video_url	VARCHAR	255	-	-	URL de la vidéo associée
price	DECIMAL	10,2	-	-	Prix individuel de la leçon
position	INT	11	NOT NULL	-	Position/ordre de la leçon dans le cours
created_at	DATETIME	-	NOT NULL	-	Date/heure de création
updated_at	DATETIME	-	NOT NULL	-	Date/heure de dernière modification

Clés étrangères : course_id → course(id)

Appartient à Course.

DOSSIER PROFESSIONNEL (DP)

Table Purchase

Cette table représente une commande effectuée par un user.

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	11	PRIMARY KEY, NOT NULL	-	Identifiant unique de l'achat
user_id	INT	11	FOREIGN KEY, NOT NULL	-	Référence à la table user
created_by_id	INT	11	NOT NULL	-	ID de l'utilisateur créateur
updated_by_id	INT	11	NOT NULL	-	ID de l'utilisateur ayant modifié
amount	DECIMAL	10,2	NOT NULL	-	Montant total de l'achat
stripe_payment_intent_id	VARCHAR	255	NOT NULL	-	ID du paiement Stripe
status	VARCHAR	50	NOT NULL	-	Statut de l'achat (pending, completed, etc.)
purchased_at	DATETIME	-	NOT NULL	-	Date/heure de l'achat
created_at	DATETIME	-	NOT NULL	-	Date/heure de création
updated_at	DATETIME	-	NOT NULL	-	Date/heure de dernière modification

Clés étrangères :

- **user_id** → user(id)
- **created_by** → user(id)
- **updated_by** → user(id)

Cette table permet de gérer le cycle de vie d'un achat, depuis le panier jusqu'au paiement.

DOSSIER PROFESSIONNEL (DP)

Table Item

Cette table détaille l'achat d'un user.

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	-	PRIMARY KEY, NOT NULL	-	Identifiant unique
purchase_id	INT	11	FOREIGN KEY → purchase(id)	-NULL	Référence à la table purchase
lesson_id	INT	-	FOREIGN KEY → lesson(id)	-NULL	Leçon(s) achetée(s)
course_id	INT		FOREIGN KEY → course(id)	-NULL	Cursus acheté
item_id	INT	11	NOT NULL	-	ID de l'article acheté
unit_price	DECIMAL	10,2	NOT NULL	-	Prix unitaire
quantity	INT	-	NOT NULL	-	Quantité

Clés étrangères :

- **course_id** → course(id)
- **lesson_id** → lesson(id)
- **purchase_id** → purchase(id)

Cette table permet à un utilisateur d'acheter une ou plusieurs leçon(s) et/ou un ou plusieurs cursus.

Table lesson_validation

Cette table enregistre les leçons validées par un user.

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	11	PRIMARY KEY, NOT NULL	-	Identifiant unique de la validation
user_id	INT	11	FOREIGN KEY, NOT NULL	-	Référence à la table <u>user</u>
lesson_id	INT	11	FOREIGN KEY, NOT NULL	-	Référence à la table <u>lesson</u>
<u>validated_at</u>	DATETIME	-	NOT NULL	-	Date/heure de validation de la leçon

Clés étrangères :

- **user_id** → user(id)
- **lesson_id** → lesson(id)

Cette table permet de suivre la progression pédagogique réelle de l'utilisateur. Elle est essentielle car elle prouve qu'une leçon a été : Accessible, suivie et validée.

Table Certification

DOSSIER PROFESSIONNEL (DP)

Cette table enregistre les certificats obtenus par les users.

Colonne	Type	Taille	Contraintes	Valeur défaut	Description
id	INT	11	PRIMARY KEY, NOT NULL	-	Identifiant unique du certificat
user_id	INT	11	FOREIGN KEY, NOT NULL	-	Référence à la table user
course_id	INT	11	FOREIGN KEY, NOT NULL	-	Référence à la table course
certificate_number	VARCHAR	255	NOT NULL	-	Numéro unique du certificat
obtained_at	DATETIME	-	NOT NULL	-	Date/heure d'obtention du certificat

Clés étrangères :

- **user_id** → user(id)
- **course_id** → course(id)

Cette table permet d'enregistrer qu'un utilisateur a obtenu un certificat après achat et validation d'un contenu.

Règles métier

Activation d'un compte :

Un User non vérifié peut s'inscrire, accéder au formulaire d'inscription et finaliser la validation de son compte.

Il ne peut pas acheter ni cursus, ni leçons si son compte n'est pas validé.

Achat :

Les achats donne accès à du contenu si :

- Le cursus est acheté -> Accès à toutes les leçons de celui-ci
- Une leçon -> Accès à la leçon achetée

Validation d'une leçon :

Après avoir suivi une leçon, l'utilisateur peut la valider grâce à un bouton. Validation enregistrée en DB. Suite à la validation, il y a une vérification pour savoir si toutes les leçons qui sont liées au cursus ont été validées ou non.

Si toutes les leçons de ce dit cursus ont été validées, alors le cursus est considéré comme validé et cela entraine une certification pour celui-ci.

Certification :

Pour obtenir une certification il faut :

- Avoir acheté le contenu nécessaire (Toutes les leçons d'un cursus)
- Valider toutes les leçons de cursus

3. La base de données dans Symfony

Pour structurer la base de données Symfony utilise **Doctrine ORM** comme couche d'abstraction pour gérer les bases de données de manière orientée objet. Au lieu de travailler directement avec des requêtes SQL, on crée des **entités** (classes PHP) qui représentent les tables de la base de données. Chaque propriété de l'entité correspond à une colonne, et les relations entre tables sont définies via des annotations ou des attributs PHP.

Cela présente des avantages :

- **Abstraction de la base de données** : Doctrine permet de changer de SGBD (MySQL, PostgreSQL, SQLite) sans modifier le code métier
- **Typage fort** : Les entités sont des classes PHP avec typage, ce qui offre une meilleure sécurité et autocomplétion dans l'IDE
- **Migrations automatiques** : Doctrine génère les migrations SQL à partir des modifications des entités, évitant les erreurs manuelles
- **Requêtes sécurisées** : Le QueryBuilder et le DQL (Doctrine Query Language) protègent contre les injections SQL
- **Relations gérées automatiquement** : Les associations (OneToMany, ManyToMany, etc.) sont traitées transparentement sans écrire de JOIN complexes
- **Lazy loading et eager loading** : Optimisation des performances en chargeant les données relationnelles à la demande ou par anticipation
- **Lifecycle callbacks** : Possibilité de déclencher des actions automatiquement (timestamps, validations) lors de la création/modification d'entités
- **Tests simplifiés** : Facile de créer des fixtures et de tester la logique métier indépendamment de la DB

Exemple d'Entity et Repository :

→ Entity

```
<?php

namespace App\Entity;

use ...

#[ORM\Entity(repositoryClass: UserRepository::class)]
#[ORM\Table(name: 'user')]
#[ORM\UniqueConstraint(name: 'UNIQ_IDENTIFIER_EMAIL', fields: ['email'])]
#[UniqueEntity(fields: ['email'], message: 'Un compte existe déjà avec cet email. ')]
class User implements UserInterface, PasswordAuthenticatedUserInterface
{
    1 usage
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    3 usages
    #[ORM\Column(length: 180)]
    private ?string $email = null;

    3 usages
    #[ORM\Column]
    private array $roles = [];

    2 usages
    #[ORM\Column]
    private ?string $password = null;
```

Attributs doctrine

→ Repository (avec QueryBuilder)

```
<?php

namespace App\Repository;

use ...

13 usages  @ Mathieu P.
class PurchaseRepository extends ServiceEntityRepository
{
    no usages  @ Mathieu P.
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Purchase::class);
    }

    8 usages  @ Mathieu P.
    public function findByUser(User $user): array
    {
        return $this->createQueryBuilder( alias: 'p')
            ->where( predicates: 'p.user = :user')
            ->setParameter( key: 'user', $user)
            ->orderBy( sort: 'p.purchasedAt', order: 'DESC')
            ->getQuery()
            ->getResult();
    }
}
```

4. Sécurité de la base de données

Contexte et problématique

L'utilisation du compte `root` MySQL dans une application présente des risques de sécurité majeurs :

- X Accès non restreint à toutes les bases de données du serveur
- X Capacité de modifier la configuration du serveur MySQL
- X Possibilité de supprimer des bases de données d'autres applications
- X Non-respect du principe du moindre privilège (*Least Privilege Principle*)

Solution mise en œuvre :

Création d'un utilisateur MySQL dédié `kl_app_user` avec :

- ✓ Privilèges limités aux bases de données du projet uniquement
- ✓ Aucun accès aux autres bases de données du serveur
- ✓ Permissions strictement nécessaires au fonctionnement de l'application
- ✓ Respect des bonnes pratiques de sécurité

Pour cela un script détaillé a été mis en place pour séparer les privilèges dans la DB :

```
-- Création de l'utilisateur avec mot de passe sécurisé
CREATE USER IF NOT EXISTS 'kl_app_user'@'localhost'
IDENTIFIED BY 'Kn0wl3dg3_S3cur3!';

-- Privilège CREATE global (nécessaire pour Doctrine)
GRANT CREATE ON *.* TO 'kl_app_user'@'localhost';

-- Privilèges sur la base principale
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, INDEX, ALTER, DROP, REFERENCES
ON knowledge_learning.*
TO 'kl_app_user'@'localhost';

-- Privilèges sur la base de test
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, INDEX, ALTER, DROP, REFERENCES
ON knowledge_learning_test.*
TO 'kl_app_user'@'localhost';

-- Support des tests parallèles (pattern matching)
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, INDEX, ALTER, DROP, REFERENCES
ON `knowledge_learning_test`%.*
TO 'kl_app_user'@'localhost';

-- Application des privilèges
FLUSH PRIVILEGES;
```

DOSSIER PROFESSIONNEL (DP)

Privèges accordés

Privèlege	Portée	Justification
CREATE	Global (*.*)	Permet à Doctrine de créer des bases de test (doctrine:database:create)
SELECT, INSERT, UPDATE, DELETE	knowledge_learning.*	Opérations CRUD sur les données
CREATE, ALTER, DROP	knowledge_learning.*	Gestion du schéma (migrations Doctrine)
INDEX	knowledge_learning.*	Optimisation des performances (index)
REFERENCES	knowledge_learning.*	Contraintes de clés étrangères

Récapitulatif du moindre privilège

Capacité	root	kl_app_user
Créer des bases de données	✓	✓ (limité)
Lire/écrire dans knowledge_learning	✓	✓
Lire/écrire dans d'autres bases	✓	X
Modifier les utilisateurs MySQL	✓	X
Arrêter le serveur MySQL	✓	X
Accéder aux tables système (mysql.*)	✓	X

Conclusion

La mise en place d'un utilisateur MySQL dédié avec des privilèges restreints permet de :

1. **Réduire la surface d'attaque** : Limitation des capacités en cas de compromission
2. **Isoler les données** : Protection des autres bases de données du serveur
3. **Traçabilité** : Identification claire de l'application accédant aux données
4. **Conformité** : Respect des normes de sécurité et des bonnes pratiques

2. Précisez les moyens utilisés :

Technologies :

- MySQL 8.0
- ORM Doctrine
- Symfony 7.4
- PHP 8.2.*
- PHPUnit

Outils :

- PhpStorm
- Git
- Github

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre européen de formation*

Chantier, atelier, service ► Projet de formation

Période d'exercice ► Du : *Cliquez ici* au : *Cliquez ici*

5. Informations complémentaires (facultatif)

Lien github : https://github.com/MathP-dev/knowledge_learning

Bilan :

La conception de cette base de données m'a permis de comprendre que **l'architecture d'une base de données constitue les fondations critiques de toute application**. Ce qui semblait initialement être une simple tâche de création de tables s'est révélé être un processus stratégique complexe. Un modèle logique de données bien conçu agit comme une **infrastructure structurante** : lorsqu'il est correctement élaboré, il garantit une scalabilité et une maintenabilité optimales ; inversement, une architecture mal pensée entraîne des problématiques de performance et des requêtes inefficaces.

J'ai également découvert la pertinence de **Doctrine ORM en tant que solution d'abstraction robuste**. L'utilisation des **attributs PHP 8** élimine les ambiguïtés liées aux annotations traditionnelles, offrant une syntaxe explicite et une intégration native avec les IDE. Les fonctionnalités telles que les migrations automatiques, la gestion transparente des relations, et la prévention des injections SQL représentent des avantages significatifs qui libèrent les ressources développeur pour se concentrer sur la logique métier plutôt que sur les préoccupations techniques de bas niveau.

En synthèse, **concevoir une base de données revient à adopter une approche orientée objet plutôt que relationnelle traditionnelle**, ce qui redéfinit substantiellement la méthodologie de développement en Symfony et favorise un code plus cohérent et maintenable.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n° 1 ► Développer la partie back-end d'une application web ou web mobile

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

1. Résumé de la mission

L'exemple précédent présentait la modélisation et la création de la base de données du projet Knowledge Learning. Pour faire suite à cet exemple, je vais vous présenter la partie back-end qui intègre et exploite cette base de données.

L'objectif pour lequel j'ai été missionné est de concevoir une plateforme « e-learning » afin de moderniser l'offre de service de cette société éditrice Knowledge, principalement connue pour ses supports de formation vendus en librairie.

Voici la liste des fonctionnalités souhaitées par l'éditeur pour son site :

- L'utilisateur peut créer un compte qu'il devra activer par mail.
- L'utilisateur peut se connecter à son compte.
- Un utilisateur qui n'a pas activé son compte ne peut acheter de cursus ou de leçon.
- L'utilisateur ayant un rôle administrateur pourra accéder au backoffice pour gérer les comptes utilisateurs, les contenus, les achats.
- L'utilisateur ayant un rôle client pourra acheter un cursus entier ou une leçon spécifique.
- Acheter une leçon ou un cursus signifie obtenir les droits d'accès aux pages des leçons concernées par l'achat.
- L'utilisateur ayant un rôle client pourra obtenir une certification « Knowledge Learning » s'il a validé toutes les leçons d'un thème.

Dans cet exemple je vais vous présenter l'architecture du projet et du code avec Symfony, le développement d'une mini-API, ainsi que la structure de tests mis en place dans le projet.

Architecture du Code

Le framework Symfony est utilisé afin de structurer l'application selon une architecture MVC (Modèle – Vue – Contrôleur).

Dans Symfony, l'architecture MVC (Modèle-Vue-Contrôleur) sépare clairement les responsabilités :

le **Modèle** gère les données et la logique métier,

la **Vue** s'occupe de l'affichage,

le **Contrôleur** fait le lien en recevant les requêtes et en orchestrant les actions.

Cette séparation améliore la lisibilité du code, facilite la maintenance, rend les tests plus simples et permet à plusieurs développeurs de travailler en parallèle sans se gêner. C'est plus organisé, plus robuste et plus scalable.

C.1 Patterns Utilisés

1. Action Controller Pattern

- Un contrôleur = une action (`__invoke()`)
- Injection de dépendances via le constructeur
- Exemple :

```
#[Route('/', name: 'app_home')]
class HomeController extends AbstractController
{
    /**
     * Mathieu P.
     */
    public function __invoke(CourseService $courseService): Response
    {
        $themes = $courseService->getAllThemes();

        return $this->render('home/index', [
            'themes' => $themes,
        ]);
    }
}
```

2. Service Layer Pattern

- Services readonly (immutables)
- Logique métier isolée des contrôleurs
- Exemple :

```
readonly class CourseService
{
    ⚙ Mathieu P.
    public function __construct(
        private CourseRepository $courseRepository,
        private ThemeRepository $themeRepository
    ) {
    }

    2 usages ⚙ Mathieu P.
    public function getAllThemes(): array
    {
        return $this->themeRepository->findAllThemes();
    }
}
```

3. Repository Pattern (Doctrine ORM)

- Séparation de la couche d'accès aux données
- Requêtes personnalisées optimisées

```
class CourseRepository extends ServiceEntityRepository
{
    ⚙ Mathieu P.
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Course::class);
    }

    2 usages ⚙ Mathieu P.
    public function findBySlug(string $slug): ?Course
    {
        return $this->createQueryBuilder( alias: 'c')
            ->leftJoin( join: 'c.theme', alias: 't')
            ->addSelect( select: 't')
            ->leftJoin( join: 'c.lessons', alias: 'l')
            ->addSelect( select: 'l')
            ->where( predicates: 'c.slug = :slug')
            ->setParameter( key: 'slug', $slug)
            ->getQuery()
            ->getOneOrNullResult();
    }
}
```

4. DTO Pattern (Data Transfer Object)

- Validation des données entrantes
- Séparation entités/formulaires

```
class RegistrationDTO
{
    2 usages
    #[Assert\NotBlank(message: 'Le prénom est obligatoire.')]
    #[Assert\Length(min: 2, max: 100)]
    private ?string $firstName = null;

    2 usages
    #[Assert\NotBlank(message: 'Le nom est obligatoire. ')]
    #[Assert\Length(min: 2, max: 100)]
    private ?string $lastName = null;

    2 usages
    #[Assert\NotBlank(message: 'L\'email est obligatoire.')]
    #[Assert\Email(message: 'L\'email n\'est pas valide.')]
    private ?string $email = null;

    2 usages
    #[Assert\NotBlank(message: 'Le mot de passe est obligatoire.')]
    #[Assert\Length(min: 8, minMessage: 'Le mot de passe doit contenir au moins 8 caractères.')]
    #[Assert\Regex(
        pattern: '/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/',
        message: 'Le mot de passe doit contenir au moins une majuscule, une minuscule et un chiffre.'
    )]
    private ?string $password = null;
```

Résumé des avantages de chaque pattern :

- **Action Controller pattern** : chaque action est isolée dans une classe dédiée (souvent invocable), ce qui rend le code plus lisible, testable et facile à maintenir. On évite les gros contrôleurs « fourre-tout ».
- **Service Layer pattern** : centralise la logique métier dans des services, ce qui évite de la disperser dans les contrôleurs ou entités. Résultat : meilleure réutilisabilité, tests simplifiés, et séparation nette entre web et métier.
- **Repository pattern** : encapsule l'accès aux données dans des classes dédiées. On découple la logique métier de la persistance, ce qui rend le code plus propre, flexible et plus simple à faire évoluer (ou à mocker en tests).
- **DTO pattern** : permet de transporter des données de façon structurée entre couches sans exposer les entités. On gagne en clarté, validation, sécurité et stabilité des échanges.

Sécurité

Mesures Implémentées

1. Protection CSRF

- Token CSRF automatique sur tous les formulaires (Symfony)
- Validation côté serveur

2. Hashage des Mots de Passe

- Algorithme **bcrypt** (Symfony PasswordHasher)
- Salage automatique (Valeur aléatoire différente pour chaque mot de passe, hash différent même si il y a des mots de passe identique)

3. Validation des Mots de Passe

- Minimum 8 caractères
- Au moins 1 majuscule, 1 minuscule, 1 chiffre
- Validation via Assert\Regex

```
#[Assert\Regex(  
    pattern: '/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/',  
    message: 'Le mot de passe doit contenir majuscule, minuscule et chiffre.'  
)]  
public ?string $password = null;
```

4. Gestion des Rôles

Pour assurer la sécurité de l'application et contrôler les différents niveaux d'accès, un système de rôles a été mis en place à l'aide du composant Symfony Security. Deux types de profils utilisateurs sont distingués dans l'application :

- ROLE_USER : accès utilisateur standard
- ROLE_ADMIN : accès backoffice EasyAdmin
- Hiérarchie : ROLE_ADMIN hérite de ROLE_USER

```
# config/packages/security.yaml  
access_control:  
    - { path: ^/admin, roles: ROLE_ADMIN }  
    - { path: ^/mon-compte, roles: ROLE_USER }  
    - { path: ^/cursus/.*acheter, roles: ROLE_USER }
```

5. Activation par Email

- Token UUID v4 généré à l'inscription
- Lien d'activation unique et temporaire

6. CSP avec NelmioSecurityBundle

```
nelmio_security:
  content_security_policy:
    defaults:
      - 'self'
```

Résumé :

Dans mon projet Symfony, j'ai mis en place plusieurs mesures de sécurité côté back-end. J'utilise la protection CSRF native de Symfony avec génération automatique de tokens et validation serveur. Les mots de passe sont hashés avec bcrypt via le PasswordHasher, et chaque mot de passe est salé automatiquement pour garantir des hash différents même en cas de mots de passe identiques. J'impose aussi des règles de complexité (au moins 8 caractères, une majuscule, une minuscule et un chiffre) grâce à des contraintes Assert\Regex. Pour la gestion des accès, j'ai défini deux rôles (ROLE_USER et ROLE_ADMIN) avec une hiérarchie où l'admin hérite des droits utilisateur, notamment pour l'accès au backoffice EasyAdmin. Enfin, j'ai mis en place une activation par email via un token UUID v4 et un lien unique et temporaire, ainsi qu'une CSP via NelmioSecurityBundle pour renforcer la sécurité globale.

API Platform

Cette mini-API a été construite avec API Platform pour exposer des données “publiables” sans donner accès directement aux entités Doctrine. La ressource exposée est un DTO (LessonOutputDto) alimenté par un Provider (LessonOutputProvider) qui récupère et transforme les entités Lesson.

L’API est montée sous le préfixe /api via la config routes API Platform :

```
# config/routes/api_platform.yaml
api_platform:
  resource: .
  type: api_platform
  prefix: /api
```

Cela active automatiquement la doc OpenAPI/Swagger et la génération des endpoints déclarés avec #[ApiResource].

La ressource publique n’est pas une entité Doctrine, mais un DTO.

C’est ce DTO qui définit les routes exposées :

```
<?php
namespace App\DTO\Api;

use ApiPlatform\Metadata\ApiProperty;
use ApiPlatform\Metadata\ApiResource;
use ApiPlatform\Metadata\Get;
use ApiPlatform\Metadata\GetCollection;
use App\State\Api\LessonOutputProvider;

/**
 * @author Mathieu P.
 */
#[ApiResource(
    shortName: 'Lesson',
    operations: [
        new Get(uriTemplate: '/lessons/{id}', provider: LessonOutputProvider::class),
        new GetCollection(uriTemplate: '/lessons', provider: LessonOutputProvider::class),
    ],
    formats: ['json' => ['application/json']],
)]
final class LessonOutputDto
{
    /**
     * @author Mathieu P.
     */
    public function __construct(
        #[ApiProperty(identifiant: true)]
        public int $id,
        public string $title,
        public string $price,
        public CourseSummaryDto $course,
    ) {}
}
```


Le provider remplace le mécanisme standard. Il applique pagination, tri, filtres, puis mappe chaque entité Lesson vers le DTO.

```
public function provide(Operation $operation, array $uriVariables = [], array $context = []): LessonOutputDto|array|null
{
    $filters = $context['filters'] ?? [];

    if ($operation instanceof CollectionOperationInterface) {
        $page = max(value: 1, (int)($filters['page'] ?? 1));
        $itemsPerPage = min(value: 50, max(value: 1, (int)($filters['itemsPerPage'] ?? 10)));

        $sort = $filters['sort'] ?? 'position';
        $sort = in_array($sort, ['id', 'title', 'price', 'position'], strict: true) ? $sort : 'position';

        $direction = strtoupper((string)($filters['direction'] ?? 'ASC')) === 'DESC' ? 'DESC' : 'ASC';

        $courseId = isset($filters['course']) ? (int)$filters['course'] : null;
        $minPrice = isset($filters['minPrice']) ? (float)$filters['minPrice'] : null;
        $maxPrice = isset($filters['maxPrice']) ? (float)$filters['maxPrice'] : null;

        $lessons = $this->lessonRepository->searchApiLessons(
            page: $page,
            itemsPerPage: $itemsPerPage,
            sort: $sort,
            direction: $direction,
            courseId: $courseId,
            minPrice: $minPrice,
            maxPrice: $maxPrice
        );

        return array_map(fn (Lesson $lesson) => $this->map($lesson), $lessons);
    }
}
```

Le mapping final vers DTO est fait ici :

```
private function map(Lesson $lesson): LessonOutputDto
{
    $course = $lesson->getCourse();

    return new LessonOutputDto(
        id: (int)$lesson->getId(),
        title: (string)$lesson->getTitle(),
        price: (string)$lesson->getPrice(),
        course: new CourseSummaryDto(
            id: (int)$course->getId(),
            title: (string)$course->getTitle(),
            description: (string)$course->getDescription(),
            price: (string)$course->getPrice(),
        )
    );
}
```

Le repository applique les filtres et le tri en SQL/Doctrine :

```
public function searchAplissons(
    int $page = 1,
    int $itemsPerPage = 10,
    string $sort = 'position',
    string $direction = 'ASC',
    ?int $courseId = null,
    ?float $minPrice = null,
    ?float $maxPrice = null
): array {
    $page = max($page, 1);
    $itemsPerPage = min(50, max($itemsPerPage, 1));

    // Whitelist anti-injection sur le champ de tri
    $allowedSorts = [
        'id' => 'l.id',
        'title' => 'l.title',
        'price' => 'l.price',
        'position' => 'l.position',
    ];
    $sortField = $allowedSorts[$sort] ?? $allowedSorts['position'];

    $direction = strtolower($direction) === 'desc' ? 'DESC' : 'ASC';

    $qb = $this->createQueryBuilder( alias: 'l' )
        ->leftJoin( join: 'l.course', alias: 'c' )
        ->addSelect( select: 'c');

    if ($courseId !== null) {
        $qb->andWhere('c.id = :courseId')
            ->setParameter( key: 'courseId', $courseId);
    }

    if ($minPrice !== null) {
        $qb->andWhere('l.price >= :minPrice')
            ->setParameter( key: 'minPrice', $minPrice);
    }

    if ($maxPrice !== null) {
        $qb->andWhere('l.price <= :maxPrice')
            ->setParameter( key: 'maxPrice', $maxPrice);
    }

    return $qb
        ->orderBy($sortField, $direction)
        ->setFirstResult( ($page - 1) * $itemsPerPage )
        ->setMaxResults($itemsPerPage)
        ->getQuery()
        ->getResult();
}
```

Sécurité et CORS

L'API est stateless, le serveur ne garde pas d'état de session entre deux requêtes :

```
# config/packages/api_platform.yaml
api_platform:
    defaults:
        stateless: true
```

Aucune règle access_control n'est définie pour /api, donc l'API est publique.

CORS (Cross-Origin Resource Sharing) est activé globalement via Nelmio :

```
# config/packages/nelmio_cors.yaml
nelmio_cors:
    defaults:
        allow_origin: ['%env(CORS_ALLOW_ORIGIN)%']
        allow_methods: ['GET', 'OPTIONS', 'POST', 'PUT', 'PATCH', 'DELETE']
        allow_headers: ['Content-Type', 'Authorization']
```

API Platform génère automatiquement la doc :

Swagger UI : /api/docs

OpenAPI JSON : /api/docs.json

La doc reflète directement les opérations définies sur LessonOutputDto et le schéma du DTO.

Conclusion

L'API est volontairement minimaliste et contrôlée : on expose un DTO et non les entités, avec un provider sur mesure qui gère le tri, les filtres et la pagination.

La doc est auto-générée via API Platform, et l'API est publique par défaut (pas d'auth).

C'est une base propre pour montrer la "construction d'une API" sans impacter le reste du projet.

Tests Unitaires et Fonctionnels

Couverture des Tests

Composant	Type	Fichier	Statut
Inscription	Unitaire	RegistrationServiceTest.php	✓
Vérification email	Unitaire	VerificationServiceTest.php	✓
Connexion	Fonctionnel	LoginTest.php	✓
Achats	Unitaire + Fonctionnel	PurchaseServiceTest.php, PurchaseFlowTest.php	✓
Repositories	Unitaire	UserRepositoryTest.php, CourseRepositoryTest.php, PurchaseRepositoryTest.php	✓

Exemple de Test Unitaire

```
class PurchaseServiceTest extends TestCase
{
    no usages new *
    public function testCreatePurchaseForCourse(): void
    {
        // Arrange
        $user = $this->createUser();
        $course = $this->createCourse();

        // Act
        $purchase = $this->purchaseService->createPurchase(
            $user, 'pi_test_123', '50.00', $course, null
        );

        // Assert
        $this->assertInstanceOf(Purchase::class, $purchase);
        $this->assertEquals('50.00', $purchase->getAmount());
        $this->assertEquals('completed', $purchase->getStatus());
    }
}
```

```
$ php bin/phpunit
PHPUnit 9.6.31 by Sebastian Bergmann and contributors.

..... 31 / 31 (100%)

Time: 00:08.589, Memory: 96.00 MB

OK (31 tests, 91 assertions)
```

Présentation du test fonctionnel LoginTest

Ce test valide le parcours de connexion/déconnexion côté utilisateur. Il vérifie l'accessibilité de la page de login, les cas de connexion correcte/incorrecte, et la déconnexion.

1) La page de connexion est accessible

Objectif : vérifier que la page /connexion s'affiche et contient les champs attendus.

```
$crawler = $client->request( method: 'GET', uri: '/connexion');

$this->assertResponseIsSuccessful();
$this->assertSelectorTextContains( selector: 'h2', text: 'Connexion');
$this->assertCount( expectedCount: 1, $crawler->filter( selector: 'input[name="login[email]"]'));
$this->assertCount( expectedCount: 1, $crawler->filter( selector: 'input[name="login[password]"]'));
```

2) Connexion avec identifiants valides

Objectif : s'assurer qu'un utilisateur vérifié peut se connecter.

```
$email = 'test.login' . uniqid() . '@example.com';
$user = $this->createVerifiedUser($email, password: 'Password123!');

$crawler = $client->request( method: 'GET', uri: '/connexion');

$form = $crawler->selectButton( value: 'Se connecter')->form([
    'login[email]' => $email,
    'login[password]' => 'Password123!',
]);

$client->submit($form);

$this->assertResponseRedirects();
$client->followRedirect();
$this->assertResponseIsSuccessful();
```

3) Connexion avec mot de passe invalide

Objectif : vérifier qu'un mauvais mot de passe est rejeté.

```
$form = $crawler->selectButton( value: 'Se connecter')->form([
    'login[email]' => $email,
    'login[password]' => 'WrongPassword',
]);

$client->submit($form);

$this->assertResponseRedirects( expectedLocation: '/connexion');
$client->followRedirect();
$this->assertSelectorExists( selector: '.alert-danger');
```

4) Connexion avec utilisateur inexistant

Objectif : vérifier le comportement quand l'email n'existe pas.

```
$form = $crawler->selectButton( value: 'Se connecter')->form([
    'login[email]' => 'nonexistent@example.com',
    'login[password]' => 'Password123!',
]);
```

5) Déconnexion

Objectif : s'assurer qu'un utilisateur connecté peut se déconnecter et perd l'accès aux pages protégées.

```
$client->loginUser($user);

$client->request( method: 'GET', uri: '/mon-compte');
$this->assertResponseIsSuccessful();

$client->request( method: 'GET', uri: '/logout');
$this->assertResponseRedirects();

$client->request( method: 'GET', uri: '/mon-compte');
$this->assertResponseRedirects( expectedLocation: '/connexion');
```

Helper : création d'utilisateur vérifié

Le test utilise un helper pour créer un utilisateur validé rapidement :

```
$user = new User();
$user->setFirstName( firstName: 'Test');
$user->setLastName( lastName: 'User');
$user->setEmail($email);
$user->setPassword($passwordHasher->hashPassword($user, $password));
$user->setVerified( isVerified: true);

$entityManager->persist($user);
$entityManager->flush();

return $user;
```

2. Précisez les moyens utilisés :

Technologies : PHP 8+, Symfony 7.4, ORM Doctrine, Twig, MySQL, Nelmio, Security bundle, Symfony CLI, PHPUnit

Outils : WSL, PhpStorm, Git, Github, Xampp, PhpmyAdmin, Yarn, MailPit, Composer

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul.

DOSSIER PROFESSIONNEL (DP)

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre européen de formation*

Chantier, atelier, service ► **Projet de formation**

Période d'exercice ► Du : [Cliquez ici](#) au : [Cliquez ici](#)

5. Informations complémentaires (facultatif)

Bilan :

Le développement du back-end avec Symfony m'a confronté à plusieurs défis techniques et méthodologiques. Mettre en place une architecture claire m'a obligé à structurer le code proprement (services, contrôleurs, entités, DTO), à organiser le projet pour qu'il reste maintenable, et à appliquer les bonnes pratiques de séparation des responsabilités.

La construction d'une API REST m'a appris à gérer la cohérence des routes, des réponses, de la validation des données et des codes HTTP, tout en veillant à la clarté de la documentation pour faciliter l'intégration.

Sur la partie sécurité, j'ai dû intégrer des mécanismes concrets (CSRF, hashage, rôles, activation, CSP), ce qui m'a sensibilisé aux risques réels et à l'importance d'anticiper les attaques dès la conception.

Enfin, la mise en place de tests m'a permis d'améliorer la fiabilité du code, de limiter les régressions et de mieux structurer mon développement autour de scénarios vérifiables.

Au final, ce projet m'a appris à concevoir un back-end complet, sécurisé et testable, tout en adoptant une démarche plus rigoureuse et professionnelle.

Lien repository : https://github.com/MathP-dev/knowledge_learning

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n° 1 ► Élaborer et mettre en œuvre des composants de gestion dans une application e-commerce

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour faire suite aux 2 exemples précédents, je vais vous présenter la mise en place d'un dashboard Administrateur et CRUD.

J'ai mis en place EasyAdmin pour gagner du temps et sécuriser l'administration de la plateforme.

L'interface me permet de gérer rapidement les entités clés (utilisateurs, thèmes, cours, leçons, achats) sans recréer toute une UI métier. Ça me donne un back-office propre, homogène et directement connecté à la base de données.

C'est un vrai accélérateur de productivité.

Je peux ajouter, modifier ou consulter des données en quelques clics, ce qui facilite le suivi du contenu pédagogique et des achats. Ça me permet aussi d'itérer plus vite côté produit, sans dépendre d'un développement front-end spécifique pour chaque besoin admin.

Côté gestion, c'est plus clair et plus fiable.

J'ai organisé les sections du CRUD avec des menus explicites et un dashboard d'accueil. Ça apporte une meilleure lisibilité pour l'administration, et ça réduit les erreurs lors de la manipulation des données.

Implémentation

1) Dashboard et menu centralisé

J'ai créé un contrôleur EasyAdminDashboardController qui définit le dashboard, le menu de navigation et les accès rapides vers chaque CRUD. J'y regroupe les entités en sections (utilisateurs, contenu pédagogique, commerce) et je lie les routes vers l'admin principal et le site.

```
class EasyAdminDashboardController extends AbstractDashboardController
    ⚙ Mathieu P.
    #[Route('/admin/crud', name: 'admin_crud')]
    public function index(): Response
    {
        return $this->render( view: 'admin/easyadmin_home');
    }

    no usages ⚙ Mathieu P.
    public function configureDashboard(): Dashboard
    {
        return Dashboard::new()
            ->setTitle( title: 'Knowledge Learning - CRUD')
            ->setFaviconPath( path: 'favicon.ico');
    }

    no usages ⚙ Mathieu P.
    public function configureMenuItems(): iterable
    {
        yield MenuItem::linkToDashboard( label: 'Dashboard Principal', icon: 'fa fa-home')
            ->setLinkRel( rel: 'app_admin_dashboard');

        yield MenuItem::section( label: '👤 Utilisateurs');
        yield MenuItem::linkToCrud( label: 'Utilisateurs', icon: 'fa fa-users', entityFqcn: User::class);

        yield MenuItem::section( label: '📚 Contenu pédagogique');
        yield MenuItem::linkToCrud( label: 'Thèmes', icon: 'fa fa-palette', entityFqcn: Theme::class);
        yield MenuItem::linkToCrud( label: 'Cours', icon: 'fa fa-book', entityFqcn: Course::class);
        yield MenuItem::linkToCrud( label: 'Leçons', icon: 'fa fa-file-alt', entityFqcn: Lesson::class);

        yield MenuItem::section( label: '🛒 Commerce');
        yield MenuItem::linkToCrud( label: 'Achats', icon: 'fa fa-shopping-cart', entityFqcn: Purchase::class);

        yield MenuItem::section( label: '🏠 Retour');
        yield MenuItem::linkToRoute( label: 'Dashboard Admin', icon: 'fa fa-tachometer-alt', routeName: 'app_admin_dashboard');
        yield MenuItem::linkToRoute( label: 'Voir le site', icon: 'fa fa-home', routeName: 'app_home');
        yield MenuItem::linkToLogout( label: 'Déconnexion', icon: 'fa fa-sign-out-alt');
    }
}
```

2) Page d'accueil admin personnalisée

J'ai ajouté une page d'accueil EasyAdmin (easyadmin_home.html.twig) qui sert de point d'entrée visuel. On y retrouve des cartes vers chaque gestion d'entité, ce qui rend l'admin plus intuitive.

```
{% extends '@EasyAdmin/page/content.html.twig' %}

{% block content_title %}
    <h1>Administration EasyAdmin</h1>
{% endblock %}

{% block main %}
    <div class="container-fluid">
        <div class="row">
            <div class="col-12">
                <div class="alert alert-info">
                    <h4>👋 Bienvenue dans l'interface d'administration</h4>
                    <p>Utilisez le menu de gauche pour gérer les différentes entités de votre plateforme.</p>
                    <hr>
                    <a href="{{ path('/admin') }}" class="btn btn-primary">
                        📊 Voir le tableau de bord avec statistiques
                    </a>
                </div>
            </div>
        </div>

        <div class="row mt-4">
            <div class="col-md-4">
                <div class="card mb-3">
                    <div class="card-body">
                        <h5 class="card-title">👤 Gestion des utilisateurs</h5>
                        <p class="card-text">Gérer les comptes, rôles et permissions</p>
                        <a href="{{ ea_url().setController('App\\Controller\\Admin\\UserCrudController').setAction('index') }}" class="btn btn-sm btn-primary">
                            Accéder
                        </a>
                    </div>
                </div>
            </div>
        </div>
    </div>
{% endblock %}
```

3) CRUD par entité (exemples concrets)

Chaque entité a son contrôleur CRUD dédié :

- **UserCrudController** pour les utilisateurs (tri, champs, actions personnalisées)

```
class UserCrudController extends AbstractCrudController
{
    no usages  🧑 Mathieu P.
    public static function getEntityFqcn(): string
    {
        return User::class;
    }

    no usages  🧑 Mathieu P.
    public function configureCrud(Crud $crud): Crud
    {
        return $crud
            ->setEntityLabelInSingular( label: 'Utilisateur')
            ->setEntityLabelInPlural( label: 'Utilisateurs')
            ->setPageTitle( pageName: 'index', title: '👤 Gestion des utilisateurs')
            ->setPageTitle( pageName: 'new', title: 'Créer un utilisateur')
            ->setPageTitle( pageName: 'edit', title: 'Modifier un utilisateur')
            ->setPageTitle( pageName: 'detail', title: 'Détails de l'utilisateur')
            ->setDefaultSort(['createdAt' => 'DESC'])
            ->setSearchFields(['email', 'firstName', 'lastName'])
            ->setPaginatorPageSize( maxResultsPerPage: 20);
    }
}
```

```
public function configureActions(Actions $actions): Actions
{
    return $actions
        ->add( pageName: Crud::PAGE_INDEX, actionNameOrObject: Action::DETAIL)
        ->update( pageName: Crud::PAGE_INDEX, actionName: Action::NEW, function (Action $action) {
            return $action
                ->setLabel( label: 'Créer un utilisateur')
                ->setIcon( icon: 'fa fa-user-plus');
        })
        ->update( pageName: Crud::PAGE_INDEX, actionName: Action::EDIT, function (Action $action) {
            return $action
                ->setLabel( label: 'Modifier')
                ->setIcon( icon: 'fa fa-edit');
        })
        ->update( pageName: Crud::PAGE_INDEX, actionName: Action::DELETE, function (Action $action) {
            return $action
                ->setLabel( label: 'Supprimer')
                ->setIcon( icon: 'fa fa-trash');
        });
}
```

```
public function configureFields(string $pageName): iterable
{
    yield IdField::new( propertyName: 'id', label: 'ID')
        ->onlyOnIndex();

    yield EmailField::new( propertyName: 'email', label: 'Email')
        ->setRequired( isRequired: true);

    yield TextField::new( propertyName: 'firstName', label: 'Prénom')
        ->setRequired( isRequired: true);

    yield TextField::new( propertyName: 'lastName', label: 'Nom')
        ->setRequired( isRequired: true);

    yield BooleanField::new( propertyName: 'isVerified', label: 'Compte vérifié')
        ->renderAsSwitch( isASwitch: false);

    yield DateTimeField::new( propertyName: 'createdAt', label: 'Date d\'inscription')
        ->hideOnForm()
        ->setFormat( dateFormatOrPattern: 'dd/MM/yyyy HH:mm');
}
```

- **ThemeCrudController** pour les thèmes (slug auto, description, compteur de cours)

```
class ThemeCrudController extends AbstractCrudController
{
    no usages  ⚙ Mathieu P.
    public static function getEntityFqcn(): string
    {
        return Theme::class;
    }

    no usages  ⚙ Mathieu P.
    public function configureCrud(Crud $crud): Crud
    {
        return $crud
            ->setEntityLabelInSingular( label: 'Thème')
            ->setEntityLabelInPlural( label: 'Thèmes')
            ->setPageTitle( pageName: 'index', title: '📁 Gestion des thèmes')
            ->setPageTitle( pageName: 'new', title: '📄 Créer un thème')
            ->setPageTitle( pageName: 'edit', title: '✎ Modifier un thème')
            ->setPageTitle( pageName: 'detail', title: '🔍 Détails du thème')
            ->setDefaultSort(['id' => 'DESC'])
            ->setSearchFields(['name', 'slug', 'description'])
            ->setPaginatorPageSize( maxResultsPerPage: 20);
    }
}
```

- **LessonCrudController** pour les leçons (ordre, associations, contenu)

```
class LessonCrudController extends AbstractCrudController
{
    no usages  ⚙ Mathieu P.
    public static function getEntityFqcn(): string
    {
        return Lesson::class;
    }

    no usages  ⚙ Mathieu P.
    public function configureCrud(Crud $crud): Crud
    {
        return $crud
            ->setEntityLabelInSingular( label: 'Leçon')
            ->setEntityLabelInPlural( label: 'Leçons')
            ->setPageTitle( pageName: 'index', title: '📁 Gestion des leçons')
            ->setPageTitle( pageName: 'new', title: '📄 Créer une leçon')
            ->setPageTitle( pageName: 'edit', title: '✎ Modifier une leçon')
            ->setPageTitle( pageName: 'detail', title: '🔍 Détails de la leçon')
            ->setDefaultSort(['course' => 'ASC', 'position' => 'ASC'])
            ->setSearchFields(['title', 'slug', 'content'])
            ->setPaginatorPageSize( maxResultsPerPage: 20);
    }
}
```

- **PurchaseCrudController** pour les achats (lecture seule, détails Stripe, formatage)

```
class PurchaseCrudController extends AbstractCrudController
{
    no usages  @ Mathieu P.
    public static function getEntityFqcn(): string
    {
        return Purchase::class;
    }

    no usages  @ Mathieu P.
    public function configureCrud(Crud $crud): Crud
    {
        return $crud
            ->setEntityLabelInSingular( label: 'Achat')
            ->setEntityLabelInPlural( label: 'Achats')
            ->setPageTitle( pageName: 'index', title: '📄 Historique des achats')
            ->setPageTitle( pageName: 'detail', title: 'Détails de l\'achat')
            ->setDefaultSort(['purchasedAt' => 'DESC'])
            ->setSearchFields(['user.email', 'course.title', 'stripeSessionId'])
            ->setPaginatorPageSize( maxResultsPerPage: 20);
    }
}
```

Chaque CRUD est paramétré avec un tri par défaut, des titres personnalisés, une pagination, des champs adaptés et des actions claires (création, édition, suppression ou lecture seule selon le cas).

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Technologies : PHP 8+, Symfony 7.4, ORM Doctrine, Twig, MySQL, EasyAdmin Bundle

Outils : WSL, PhpStorm, Git, Github, Xampp, PhpmyAdmin, Yarn, Composer

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul.

4. Contexte

Nom de l'entreprise, organisme ou association ► *Centre européen de formation*

Chantier, atelier, service ► *Projet de formation*

Période d'exercice ► Du : *Cliquez ici* au : *Cliquez ici*

5. Informations complémentaires (facultatif)

Bilan — Dashboard admin & CRUD

La mise en place du dashboard admin et des CRUD avec EasyAdmin a été un vrai exercice de structuration. Le principal défi a été de rendre l'administration claire et efficace tout en respectant la logique métier de l'application (utilisateurs, contenus, achats). J'ai dû organiser les menus, penser la hiérarchie des sections, et adapter chaque CRUD pour qu'il colle aux besoins réels (ex. lecture seule pour les achats, tri et recherche pour les utilisateurs, champs spécifiques pour les leçons).

Ce travail m'a appris à concevoir un back-office fonctionnel et maintenable, à personnaliser finement EasyAdmin (actions, champs, titres, navigation), et à créer une interface admin qui fait gagner du temps tout en limitant les erreurs de gestion. Au final, j'ai compris l'importance d'un back-office bien pensé pour piloter une application proprement, sans alourdir le front-end.

Activité-type 2

Développer la partie back-end d'une application web ou web mobile en intégrant les recommandations de sécurité

Exemple n° 1 ► *Expérience professionnelle en entreprise*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Au cours de mon stage, j'ai participé à plusieurs missions concrètes orientées qualité et automatisation. J'ai d'abord mis en place un générateur de présentations PowerPoint afin d'industrialiser la production de supports. J'ai également déployé des tests end-to-end avec Playwright et intégré leur exécution dans une chaîne CI/CD, ce qui a amélioré la fiabilité des livraisons et la détection des régressions. Enfin, j'ai contribué à la mise en place d'un outil de gestion dédié à la TMA (Tierce Maintenance Applicative), c'est-à-dire l'ensemble des activités de maintenance, d'évolution et de correction d'une application confiées à un prestataire, afin de mieux suivre, adapter et prioriser les demandes. (Je n'ai pas eu l'autorisation de partager la code base de cet outil)

En parallèle, j'ai aussi remis en ordre la gestion des traductions (français/anglais) pour améliorer la cohérence et la maintenabilité de l'internationalisation du projet.

Générateur de présentations PowerPoint

Pour automatiser la création de présentations, j'ai intégré le bundle **PHPOffice/PHPPresentation** au projet. Concrètement, je l'ai ajouté via Composer, puis j'ai développé un service dédié qui construit les slides dynamiquement (titres, sections, tableaux, contenus) à partir des données applicatives. J'ai ensuite exporté les fichiers en .pptx et intégré le tout dans le workflow métier.

```
public function createPowerPoint(array $data, array $summaryImputedTime): PhpPresentation
{
    // Charge le template PowerPoint "Compte rendu TMA"
    $pptReader = IOFactory::createReader('PowerPoint2007');
    $powerPoint = $pptReader->load($this->appKernel->getProjectDir() . '/data/template-TMA-v2.pptx');

    // ===== SLIDE 1 =====
    // récupère la 1ère slide du template (les INDEX commencent à 0)
    $slide1 = $powerPoint->getSlide(0);
    // ajoute du contenu à la première slide
    $this->addContentToFirstSlide($slide1);

    // ===== SLIDE 2 =====
    // récupère la 2ème slide du template (les INDEX commencent à 0)
    $slide2 = $powerPoint->getSlide(1);
    $slide2->createLineShape(65, 700, 350, 700)->getBorder()->setLineWidth(2)->getColor()->setRGB('052864');
    // ajoute du contenu à la seconde slide
    $this->addContentToSecondSlide($slide2, $data, $summaryImputedTime);

    // ===== SLIDE 3 =====
    // ajoute du contenu à la troisième slide : "le résumé"

    $projectParentAndChildren = array_reduce($data, static function ($result, $d) {
        $result[$d['project_name']][] = $d;
        return $result;
    }, []);

    $slide3 = $powerPoint->getSlide(2);
    $this->addContentToThirdSlide($slide3, $data);
    $slide3->createLineShape(65, 675, 350, 675)->getBorder()->setLineWidth(2)->getColor()->setRGB('052864');
```


DOSSIER PROFESSIONNEL (DP)

```
private function addContentToFirstSlide(Slide $slide1): void
{
    // ===== Recherche dans les data la première date trouvée et la dernière pour établir la date range
    $this->dateFromFormatted = $this->helper->getDateInFrench("%B %Y", $this->dateFrom->getTimestamp());
    $this->dateToFormatted = $this->helper->getDateInFrench("%B %Y", $this->dateTo->getTimestamp());

    $height = 66;
    $offsetX = 56;
    $offsetY = 245;
    // ===== Ajoute le nom projet et code facturation de la page d'intro =====
    $this->createTitle($slide1, $height, $offsetX, 50, 'FF000080');
    $offsetY = $this->getProjectNameAndCodeBilling($slide1, $height, $offsetX, $offsetY, 'FFFFFF', 38, true);

    $slide1->createLineShape(65, $offsetY, 250, $offsetY)->getBorder()->setColor(new StyleColor('FFFFFF'));

    // ===== Ajoute la date sur la page d'intro =====
    $offsetY += 10;
    $dateIntro = $slide1->createRichTextShape()
        ->setHeight(36)
        ->setWidth(500)
        ->setOffsetX(56)
        ->setOffsetY($offsetY)
    ;

    if ($this->dateFromFormatted !== $this->dateToFormatted) {
        $textRun = $dateIntro->createTextRun(
            ucfirst($this->dateFromFormatted) . ' à ' . ucfirst($this->dateToFormatted)
        );
    } else {
        $textRun = $dateIntro->createTextRun(ucfirst($this->dateFromFormatted));
    }

    $textRun->getFont()
        ->setName('Arial')
        ->setBold(true)
        ->setSize(20)
        ->getColor()->setRGB('052864')
    ;
}
```

```
private function createTitle(
    Slide $slide,
    $height,
    $offsetX,
    $offsetY,
): void {
    $container = $slide->createRichTextShape()
        ->setHeight($height)
        ->setWidth(570)
        ->setOffsetX($offsetX)
        ->setOffsetY($offsetY)
    ;
    $textRun = $container->createTextRun(
        "Compte rendu\nd'activité TMA"
    );
    $textRun->getFont()
        ->setName('Arial')
        ->setBold(true)
        ->setSize(40)
        ->getColor()->setRGB('052864')
    ;
}
```

Avantages : cette solution me permet de **générer des supports homogènes**, personnalisés et rapides à produire, sans dépendre d'une création manuelle. Ça **fait gagner du temps** aux équipes et réduit le risque d'erreurs ou d'incohérences dans les livrables.

Défis rencontrés :

- La **mise en page** automatique (alignements, tailles, marges) n'était pas simple à calibrer.
- La **gestion des images** et des tableaux a demandé des ajustements pour obtenir un rendu propre.
- J'ai aussi dû **gérer la stabilité** des exports (compatibilité PowerPoint).

Solutions apportées :

J'ai mis en place une **structure de slides standardisée**, défini des styles réutilisables, et créé des helpers pour centraliser la génération des éléments (titres, blocs, tableaux). J'ai également testé les exports sur plusieurs exemples pour valider le rendu final et ajuster les paramètres.

Tests End 2 End avec PlayWright

Pour fiabiliser les parcours utilisateurs, nous avons mis en place des tests E2E avec Playwright. Nous avons standardisé l'exécution via un **Makefile**, un **script bash de setup**, et un **docker-compose dédié**.

Le workflow est le suivant :

1. un conteneur Postgres dédié aux tests est lancé,

```
docker-compose.e2e.yaml
1 services:
2   e2e-docker-makefile:
3     image: postgres:14-alpine
4     environment:
5       POSTGRES_USER: [REDACTED]
6       POSTGRES_PASSWORD: [REDACTED]
7       POSTGRES_DB: suivi_temps_test
8     healthcheck:
9       test: ['CMD-SHELL', 'pg_isready -q -h e2e-docker-makefile']
10      interval: 3s
11      timeout: 1s
12      retries: 10
13     networks:
14       - test_network
15
16   executor:
17     image: [REDACTED]
18     command: bash /app/setup-e2e.sh
19     environment:
20       APP_ENV: test
21       OPEN_REPORT: never
22     working_dir: /app
23     volumes:
24       - ./app
25       - ./env.test:/app/.env.test
26     depends_on:
27       e2e-docker-makefile:
28         condition: service_healthy
29     networks:
30       - test_network
31
32 networks:
33   test_network:
```

2. la base est recréée et réinitialisée,

```
Makefile
1 prepare_e2e_db:
2     docker compose -f docker-compose.e2e.yaml up -d e2e-docker-makefile
3     docker compose -f docker-compose.e2e.yaml exec -T e2e-docker-makefile bash -c "until pg_isready -U root -h localhost -p 5432; do sleep 1; done"
4     docker compose -f docker-compose.e2e.yaml exec -T e2e-docker-makefile dropdb -U root --if-exists suivi_temps_test
5     docker compose -f docker-compose.e2e.yaml exec -T e2e-docker-makefile createdb -U root suivi_temps_test
6     docker compose -f docker-compose.e2e.yaml exec -T e2e-docker-makefile psql -U root -d suivi_temps_test < db/initialDatabase.sql
7     docker login
8
9 e2e_tests: prepare_e2e_db
10     docker compose -f docker-compose.e2e.yaml up --abort-on-container-exit executor
11
12 cleanup_tests:
13     docker compose -f docker-compose.e2e.yaml down -v
14
15 e2e: e2e_tests cleanup_tests
```

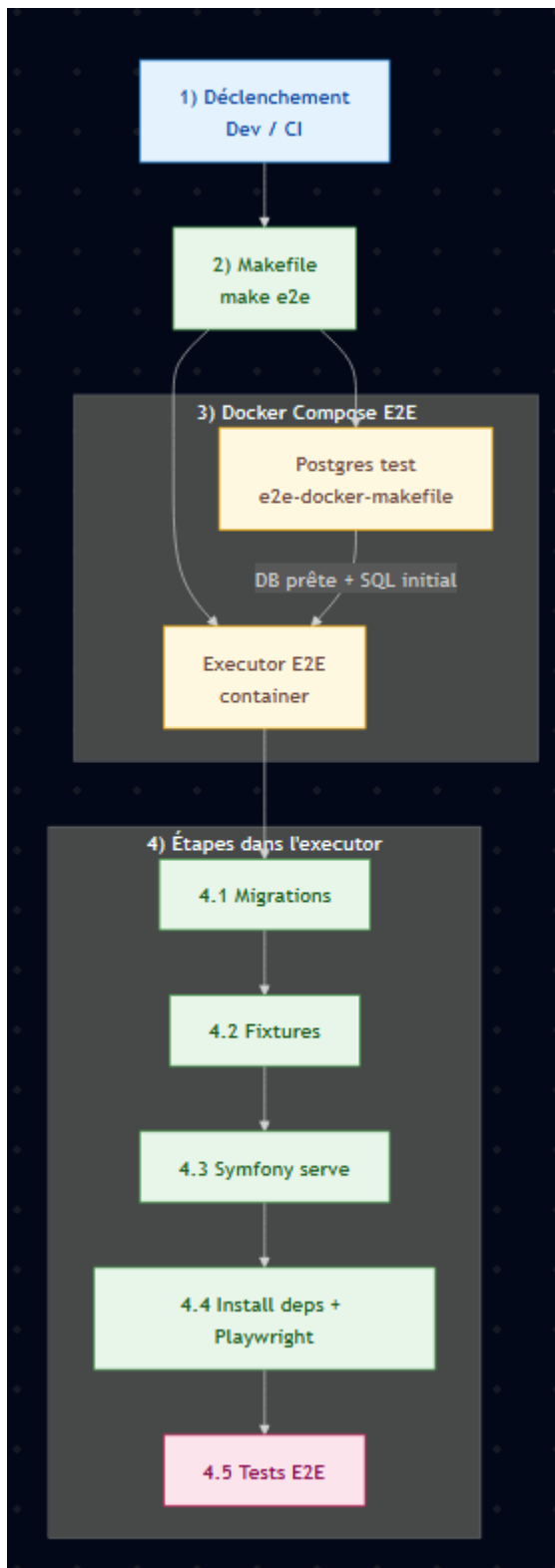
3. un conteneur “executor” prépare l’environnement (migrations + fixtures + dépendances JS + Playwright),

```
setup-e2e.sh
1 #!/bin/sh
2 set -e
3
4 # Migrations et fixtures dans le container
5 symfony console doctrine:migrations:migrate --no-interaction --env=test
6 symfony console doctrine:fixtures:load --no-interaction --env=test
7 symfony proxy:start
8 symfony serve --no-workers -d
9
10 # Install dépendances JS
11 yarn install
12 apt-get update --allow-releaseinfo-change
13 yarn playwright install --with-deps
14
15 # Execute tests E2E Playwright
16 yarn e2e
17
18 echo "Tests done!"
```

4. les tests sont exécutés automatiquement.

Cela rend l’exécution **reproductible**, **isolée**, et **compatible CI/CD**.

Architecture E2E :



Défis rencontrés & solutions

Défi 1 : Préparer une base propre à chaque run

→ *Solution* : j'ai automatisé la création/suppression de la base et le chargement d'un SQL initial via Makefile.

Défi 2 : Standardiser l'environnement local / CI

→ *Solution* : j'ai tout encapsulé dans Docker + script bash, pour éviter "ça marche sur ma machine".

Défi 3 : Dépendances Playwright et navigateurs

→ *Solution* : installation Playwright + deps dans le conteneur (yarn playwright install --with-deps).

Avantages de la mise en place de tests E2E

Qualité renforcée : je vérifie les parcours réels (connexion, navigation, actions métier).

Reproductibilité : même environnement local/CI grâce à Docker.

Gain de temps : une seule commande lance tout le cycle (DB → setup → tests → nettoyage).

Fiabilité en livraison : les régressions sont détectées tôt.

DOSSIER PROFESSIONNEL (DP)

2. Précisez les moyens utilisés :

Technologies et Outils: PHP, Symfony, POSTGreSQL, Docker, Playwright, Git, Gitlab, PHPOffice/PHPPresentation

3. Avec qui avez-vous travaillé ?

J'ai travaillé en groupe

4. Contexte

Nom de l'entreprise, organisme ou association ► *UmanIt*

Chantier, atelier, service ► Projet de stage

Période d'exercice ► Du : *01/09/2025* au : *31/11/2025*

5. Informations complémentaires (facultatif)

Bilan du stage

Ce stage m'a énormément apporté, autant sur le plan technique que professionnel. J'ai découvert le fonctionnement réel d'une équipe de développement, les contraintes de qualité, de délais, et les bonnes pratiques du monde pro. J'ai adoré cette expérience et elle a confirmé mon envie de faire carrière dans le développement : c'est un domaine qui me passionne réellement.

Au-delà des compétences techniques, j'ai aussi gagné en rigueur, en autonomie et en organisation. J'ai appris à livrer des fonctionnalités concrètes, à tester, à documenter, et à travailler dans un environnement structuré. Mon maître de stage a été très satisfait de notre collaboration et a souligné mon implication et la qualité de mon travail, malgré mon profil encore junior. Cette reconnaissance m'a motivé et conforté dans mon choix de poursuivre dans ce secteur.

DOSSIER PROFESSIONNEL (DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussigné(e) Mathieu Paquier

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis
l'auteur(e) des réalisations jointes.

Fait à Nantes le 15/04/2026

pour faire valoir ce que de droit.

Signature :



DOSSIER PROFESSIONNEL (DP)

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

DOSSIER PROFESSIONNEL (DP)

ANNEXES

(Si le RC le prévoit)